

SPOOFTOOPH

The Complete Ethical Hacker's Guide to Bluetooth Device Spoofing in Kali Linux

Bluetooth Reconnaissance · Identity Spoofing · Authorized Penetration Testing

Spooftooph: The Complete Ethical Hacker's Guide to Bluetooth Device Spoofing in Kali Linux

First Edition, 2024

Author: Tanish Sareen

Institution: CT University — BCA Cyber Security & Forensics



Copyright © 2024 Tanish Sareen. All rights reserved.

No part of this publication may be reproduced, distributed, or transmitted in any form or by any means, including photocopying, recording, or other electronic or mechanical methods, without the prior written permission of the author, except in the case of brief quotations embodied in critical reviews and certain other non-commercial uses permitted by copyright law.

Legal Disclaimer

This book is intended strictly for educational purposes and authorized security testing. The techniques, tools, and methodologies described in this work must only be applied to systems, devices, and networks for which the reader has explicit written permission. Unauthorized use of the techniques described herein may constitute a criminal offence under the Information Technology Act, 2000 (India), the Computer Fraud and Abuse Act (USA), the Computer Misuse Act (UK), and equivalent legislation in other jurisdictions. The author and publisher accept no liability for any misuse of the information contained in this book. Readers assume full legal and ethical responsibility for all activities performed based on knowledge gained from this material.

AUTHOR

Tanish Sareen

BCA Cyber Security & Forensics | CT University

First Edition · 2024

Kali Linux is a registered trademark of OffSec Services Limited. Bluetooth is a registered trademark of the Bluetooth SIG, Inc. All other trademarks mentioned in this book are the property of their respective owners. References to specific products, tools, or organisations do not constitute endorsement.

Printed and distributed for academic and professional development use.

TABLE OF CONTENTS

Preface

Chapter 1 Foundations of Bluetooth Technology

- 1.1 The Wireless Landscape and Bluetooth's Place
- 1.2 A Brief History of Bluetooth
- 1.3 Bluetooth Versions and Standards
- 1.4 Bluetooth Classic vs. Bluetooth Low Energy (BLE)
- 1.5 How Bluetooth Works: Frequency Hopping and Piconets
- 1.6 Bluetooth Protocol Stack
- 1.7 Bluetooth Profiles
- 1.8 Chapter Summary

Chapter 2 Bluetooth Security Architecture

- 2.1 Bluetooth Addressing: The BD_ADDR
- 2.2 Device Classes and CoD
- 2.3 Pairing and Bonding Mechanisms
- 2.4 Bluetooth Security Modes
- 2.5 Common Bluetooth Vulnerabilities
- 2.6 The Threat Landscape
- 2.7 Chapter Summary

Chapter 3 BlueZ: The Linux Bluetooth Stack

- 3.1 BlueZ Architecture Overview
- 3.2 Core Components and Daemons
- 3.3 HCI – Host Controller Interface
- 3.4 D-Bus API and bluetoothctl
- 3.5 Key BlueZ Configuration Files
- 3.6 BlueZ Tools Reference
- 3.7 Chapter Summary

Chapter 4 SpoofTooph: History, Background, and Design

- 4.1 Origins and Development History
- 4.2 Purpose and Design Philosophy
- 4.3 How SpoofTooph Modifies Bluetooth Identity
- 4.4 SpoofTooph in the Kali Linux Ecosystem
- 4.5 Limitations and Scope
- 4.6 Chapter Summary

Chapter 5 Installation and Environment Setup

- 5.1 Hardware Requirements
- 5.2 Installing Spooftooph on Kali Linux
- 5.3 Installing on Debian/Ubuntu Systems
- 5.4 Verifying Bluetooth Adapter Support
- 5.5 Configuring the Lab Environment
- 5.6 Virtual Machine Considerations
- 5.7 Chapter Summary

Chapter 6 Complete Command Reference

- 6.1 Spooftooph Syntax Overview
- 6.2 All Flags and Options Explained
- 6.3 Spoofing by Device Address (-a)
- 6.4 Spoofing by Device Name (-n)
- 6.5 Spoofing by Device Class (-d)
- 6.6 Scanning and Cloning (-s, -S)
- 6.7 Log File Operations (-l, -r)
- 6.8 Specifying HCI Interface (-i)
- 6.9 Combined Usage Patterns
- 6.10 Chapter Summary

Chapter 7 Bluetooth Reconnaissance Techniques

- 7.1 Passive vs. Active Reconnaissance
- 7.2 Device Discovery with hcitool
- 7.3 Service Enumeration with sdptool
- 7.4 Scanning with btscanner and Bluelog
- 7.5 Mapping the Bluetooth Environment
- 7.6 Information Gathering Workflow
- 7.7 Chapter Summary

Chapter 8 Practical Laboratory Exercises

- 8.1 Lab 1: Setting Up the Test Environment
- 8.2 Lab 2: Basic Device Discovery
- 8.3 Lab 3: Cloning a Bluetooth Identity
- 8.4 Lab 4: Spoofing Device Class
- 8.5 Lab 5: Log-File-Based Spoofing
- 8.6 Lab 6: Automated Scan-and-Clone
- 8.7 Lab 7: Bluetooth Impersonation Assessment
- 8.8 Lab 8: Capture and Analysis with Wireshark
- 8.9 Chapter Summary

Chapter 9 Tool Comparison and Assessment Ecosystem

- 9.1 Spooftooph vs. hciconfig

9.2 Spooftooph vs. btlejuice / GATTacker

9.3 Spooftooph vs. Bluefang

9.4 Spooftooph vs. Bettercap (BLE)

9.5 Choosing the Right Tool

9.6 Chapter Summary

Chapter 10 Legal and Ethical Framework

10.1 Applicable Laws and Regulations

10.2 Written Authorisation and Scope

10.3 Responsible Disclosure

10.4 Professional Ethics Codes

10.5 Chapter Summary

Chapter 11 Defensive Strategies and Hardening

11.1 Bluetooth Security Best Practices

11.2 Detecting Spoofing Attacks

11.3 Bluetooth IDS/IPS Solutions

11.4 Enterprise Bluetooth Policy

11.5 Chapter Summary

Chapter 12 Case Studies and Security Lessons

12.1 Case Study 1: Healthcare Device Impersonation

12.2 Case Study 2: Automotive Bluetooth Assessment

12.3 Case Study 3: Smart-Home Reconnaissance

12.4 Lessons Learned

12.5 Chapter Summary

Appendix A Troubleshooting Guide

Appendix B Glossary of Terms

Appendix C References and Further Reading

PREFACE

Wireless communication has become the invisible fabric of modern civilisation. Billions of devices — headphones, keyboards, medical sensors, smartwatches, automotive systems, industrial controllers — exchange sensitive data over Bluetooth every second of every day. Yet despite three decades of development, Bluetooth security remains poorly understood even by many professionals in the industry.

This book was written to change that. It is a comprehensive, technically rigorous guide to Bluetooth device identity spoofing as practised in the context of authorised penetration testing and security research. The central tool examined is **Spooftooph** — a command-line utility included in Kali Linux that allows a security professional to manipulate the Bluetooth Device Address, device name, and Class of Device of a local Bluetooth adapter, enabling identity-based assessments against authorised targets.

The book is structured for progressive learning. Readers with no prior Bluetooth knowledge will find thorough coverage of the protocol stack, hardware, and security architecture in the early chapters. Experienced practitioners can jump directly to the command reference, laboratory exercises, and case studies. Every concept is illustrated with practical examples, and every example is framed within an ethical and legal context.

A word on responsibility: the techniques in this book are powerful. They can be used to identify serious vulnerabilities in real-world systems — and that is precisely their value. But power demands care. Use these skills only where you have written permission to do so. Contribute to a safer digital world.

— Tanish Sareen, CT University, 2024

CHAPTER 1

Foundations of Bluetooth Technology

1.1 The Wireless Landscape and Bluetooth's Place

Modern computing environments are permeated by wireless standards: Wi-Fi (IEEE 802.11) handles high-bandwidth local networking; cellular (4G/5G) provides wide-area connectivity; Zigbee and Z-Wave serve low-power IoT meshes; NFC enables centimetre-range contactless data exchange. Bluetooth occupies a distinct niche: personal-area networking at short range (typically 1–100 m), moderate data rates, and moderate power consumption.

This positioning made Bluetooth the dominant protocol for connecting peripheral devices to host computers and smartphones. Understanding its architecture — and its weaknesses — is a foundational skill for any security professional working in modern enterprise, consumer, or industrial environments.

1.2 A Brief History of Bluetooth

Bluetooth was conceived in 1994 by Jaap Haartsen and Sven Mattisson at Ericsson as a wireless alternative to RS-232 serial cables. The project was named after Harald Bluetooth (Harald Blåtand), a 10th-century Danish king credited with uniting Scandinavian tribes — a metaphor for the technology's mission to unite disparate devices.

In 1998, the Bluetooth Special Interest Group (SIG) was founded by Ericsson, Nokia, IBM, Toshiba, and Intel. The SIG remains the governing body for Bluetooth standards, which have evolved significantly through the following milestones:

Year	Milestone	Key Development
1994	Ericsson	Internal research project; cable replacement concept
1998	Bluetooth SIG founded	Ericsson, Nokia, IBM, Toshiba, Intel
1999	Bluetooth 1.0	First specification; raw 1 Mbps data rate
2001	Bluetooth 1.1	Stability fixes; first commercial wave
2003	Bluetooth 1.2	AFH (Adaptive Frequency Hopping); better coexistence with Wi-Fi
2004	Bluetooth 2.0 + EDR	Enhanced Data Rate; up to 3 Mbps; reduced power
2007	Bluetooth 2.1 + EDR	SSP (Secure Simple Pairing); ECDH key exchange

2009	Bluetooth 3.0 + HS	High Speed; 802.11 coexistence for burst transfers
2010	Bluetooth 4.0	BLE introduced; low energy mode for IoT
2013	Bluetooth 4.1	Coexistence with LTE; topology improvements
2014	Bluetooth 4.2	Privacy enhancements; LE Data Packet Extension
2016	Bluetooth 5.0	4x range, 2x speed for BLE; Bluetooth Mesh foundation
2019	Bluetooth 5.1	Direction-finding (AoA/AoD); sub-metre positioning
2020	Bluetooth 5.2	LE Audio; Isochronous Channels (ISOC)
2021	Bluetooth 5.3	Enhanced Connection Update; periodic advertising enhancements
2023	Bluetooth 5.4	Periodic Advertising with Responses (PAWR)

Table 1.1 — Bluetooth Version History

1.3 Bluetooth Versions and Standards

Each Bluetooth version builds upon its predecessors while introducing new capabilities. From a security perspective, the most significant evolution occurred at version 2.1 (SSP with ECDH), version 4.0 (introduction of BLE with distinct security model), and version 4.2 (LE Privacy). Legacy devices — still common in industrial and healthcare settings — may support only Bluetooth 2.x, which has well-documented vulnerabilities.

1.4 Bluetooth Classic vs. Bluetooth Low Energy (BLE)

Bluetooth 4.0 introduced two distinct modes: **Bluetooth Classic** (BR/EDR) and **Bluetooth Low Energy** (BLE, also called Bluetooth Smart). These modes have fundamentally different architectures, use cases, and security properties:

Feature	Bluetooth Classic (BR/EDR)	Bluetooth Low Energy (BLE)
Primary use case	Audio streaming, file transfer, HID	Sensors, beacons, wearables, IoT
Frequency	2.4 GHz ISM (79 channels, 1 MHz)	2.4 GHz ISM (40 channels, 2 MHz)
Data rate	1–3 Mbps	125 Kbps – 2 Mbps (BT 5.0)
Latency	~100 ms	~3–6 ms
Power consumption	Moderate (~1 W peak)	Very low (~0.01–0.5 W)
Range	10–100 m (class dependent)	10 m – several km (BT 5.0)

Pairing	Legacy PIN / SSP	Just Works / Passkey / OOB / Numeric Comparison
Address type	BD_ADDR (48-bit, public)	Public + Random (static/private)
Security baseline	Bluetooth 2.1+ SSP	LE Security Mode 1 / Mode 2
Spooftooph relevance	Primary target (BD_ADDR manipulation)	Partial (public address)

Table 1.2 — Bluetooth Classic vs. BLE Comparison

1.5 How Bluetooth Works: Frequency Hopping and Piconets

Bluetooth Classic uses **Frequency Hopping Spread Spectrum (FHSS)** across the 2.4 GHz ISM band, divided into 79 channels of 1 MHz each. The radio hops between channels 1,600 times per second using a pseudorandom sequence derived from the master device's clock and BD_ADDR. This provides resistance to narrowband interference and a degree of security against passive eavesdropping.

A **piconet** is the basic Bluetooth network topology: one master device and up to seven active slave devices, all synchronised to the master's clock. Multiple piconets can interconnect to form a **scatternet**, where bridge devices participate in more than one piconet simultaneously.



Figure 1.1 — Bluetooth Piconet Topology

1.6 Bluetooth Protocol Stack

The Bluetooth protocol stack is divided into a hardware layer and a host layer, separated by the Host Controller Interface (HCI). Understanding this layering is critical for security analysts, as vulnerabilities may exist at any layer.



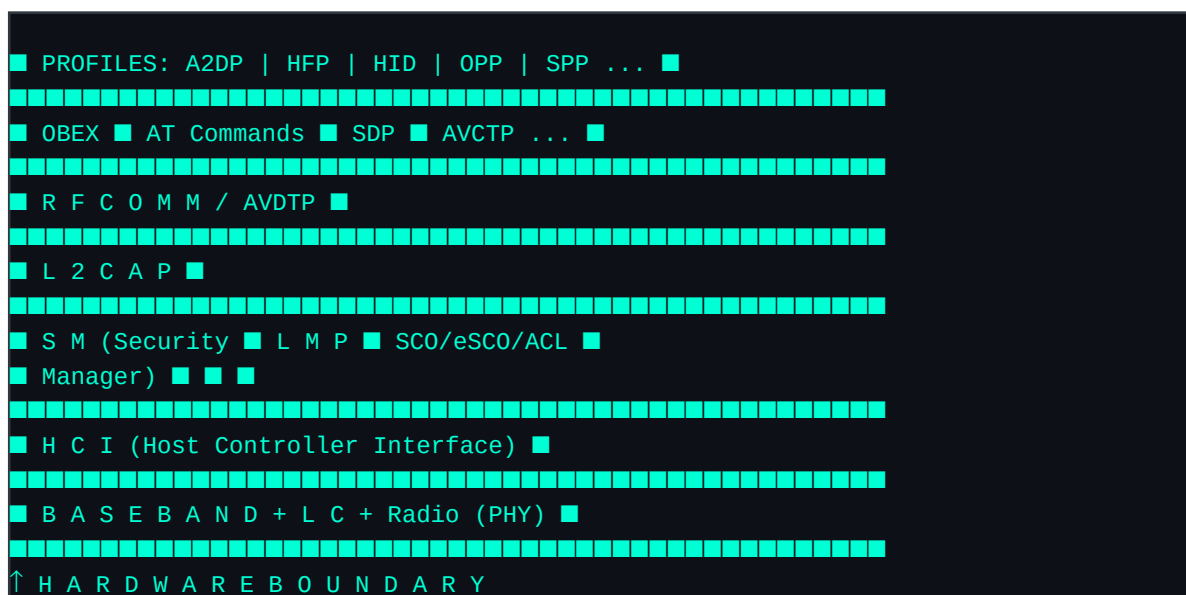


Figure 1.2 — Bluetooth Protocol Stack

Key layers from a security perspective:

- **Radio (PHY):** Physical transmission on 2.4 GHz. Implements FHSS or DSSS (BLE).
- **Baseband:** Manages channel access, packet framing, error correction, and flow control.
- **LMP (Link Manager Protocol):** Authentication, encryption negotiation, power control.
- **HCI (Host Controller Interface):** Standardised command/event interface between host software and controller firmware.
- **L2CAP (Logical Link Control and Adaptation Protocol):** Multiplexes higher-layer protocols, handles segmentation/reassembly.
- **SDP (Service Discovery Protocol):** Allows devices to discover services advertised by peers — a significant information leakage source.
- **Security Manager (SM):** Handles key generation, distribution, and authentication for BLE.
- **RFCOMM:** Emulates RS-232 serial ports over L2CAP — carries SPP, HFP, HSP.
- **Profiles:** Application-layer specifications defining interoperability for specific use cases.

1.7 Bluetooth Profiles

Bluetooth profiles define how two devices use the protocol stack for a specific application. From a penetration-testing viewpoint, active profiles reveal device functionality and potential attack surfaces.

Profile	Abbreviation	Function	Security Consideration
Advanced Audio Distribution	A2DP	Stereo audio streaming	Audio eavesdropping if link is unencrypted

Hands-Free Profile	HFP	Car kit / headset telephony	Call interception; AT command injection
Human Interface Device	HID	Keyboards, mice, gamepads	Keystroke injection; HID-over-GATT attacks
Object Push Profile	OPP	File transfer (vCards, etc.)	BlueJacking; unsolicited data push
Serial Port Profile	SPP	Serial emulation over BT	Command injection on embedded systems
File Transfer Profile	FTP	Directory browsing/transfer	Unauthorised file access (legacy devices)
Personal Area Networking	PAN	Internet sharing over BT	Network pivoting
Health Device Profile	HDP	Medical sensor data	Patient data interception; device spoofing

Table 1.3 — Common Bluetooth Profiles and Security Relevance

Chapter Summary

- ✓ Bluetooth is a short-range WPAN standard evolved from a 1994 Ericsson cable-replacement project.
- ✓ Bluetooth Classic (BR/EDR) targets high-bandwidth peripherals; BLE targets low-power IoT.
- ✓ The protocol stack separates hardware (baseband + radio) from host software via HCI.
- ✓ Profiles define application-layer interoperability and reveal device capabilities during reconnaissance.
- ✓ Security vulnerabilities may exist at every layer of the stack.

CHAPTER 2

Bluetooth Security Architecture

2.1 Bluetooth Addressing: The BD_ADDR

Every Bluetooth Classic device is assigned a unique 48-bit **Bluetooth Device Address (BD_ADDR)**. This address is structurally identical to an Ethernet MAC address and is split into three fields:

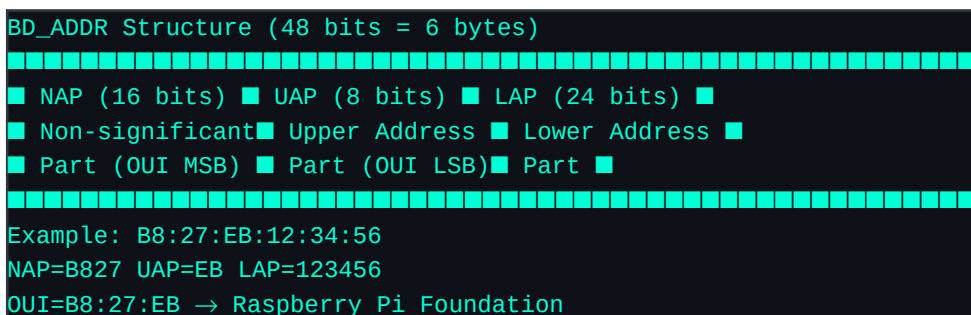


Figure 2.1 — BD ADDR Field Structure

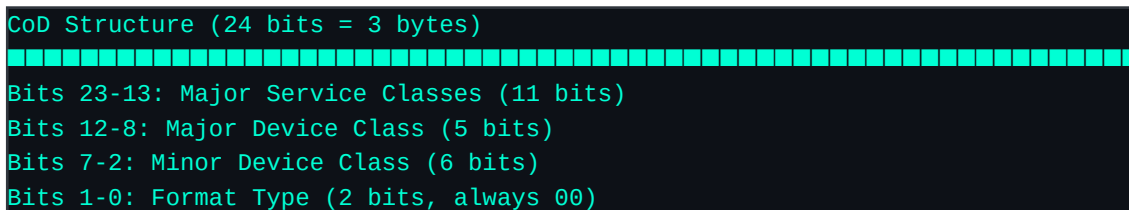
The first three bytes (NAP + UAP) form the **Organizationally Unique Identifier (OUI)**, assigned by the IEEE to manufacturers. The OUI reveals the device manufacturer, which is invaluable during reconnaissance. Publicly available OUI lookup services (e.g., IEEE's MA-L registry, Wireshark's manufacturer database) can identify the vendor from any BD ADDR.

NOTE

The BD_ADDR is the primary target for Spooftooph. By changing the local adapter's BD_ADDR, an attacker (or security tester) can impersonate a different device from the perspective of any scanning peer.

2.2 Device Classes and CoD

The **Class of Device (CoD)** is a 24-bit field that classifies a Bluetooth device by its primary service capabilities and device type. It is transmitted during inquiry responses and influences how other devices treat the advertising device.



Major Device Class	Value (hex)	Examples
--------------------	-------------	----------

Miscellaneous	0x000	Unclassified devices
Computer	0x100	Desktop, laptop, server, tablet
Phone	0x200	Smartphone, feature phone, modem
LAN/Network AP	0x300	Access point, router
Audio/Video	0x400	Headset, speaker, microphone, TV
Peripheral	0x500	Mouse, keyboard, joystick
Imaging	0x600	Printer, scanner, camera
Wearable	0x700	Wristwatch, pager, glasses
Toy	0x800	Robot, vehicle, doll, game
Health	0x900	Blood pressure, thermometer, glucose
Uncategorised	0x1F00	Catch-all

Table 2.1 — Bluetooth Major Device Classes

Spooftooph allows the security tester to set an arbitrary CoD value, enabling impersonation of any device class. For example, setting CoD to 0x002540 would present the adapter as a smartphone, while 0x000540 would present it as a laptop.

2.3 Pairing and Bonding Mechanisms

Pairing is the process by which two Bluetooth devices establish a shared secret (a link key) used for authentication and encryption. **Bonding** refers to the persistent storage of this link key for future reconnections.

Bluetooth 2.0 and Earlier — Legacy Pairing

Legacy pairing uses a PIN code (4–16 decimal digits) entered on both devices. The PIN, combined with the devices' BD_ADDRs and random numbers, derives an initialisation key via the E22 algorithm, which then generates a link key via E21. Legacy pairing is vulnerable to brute-force and dictionary attacks if the PIN is short or predictable (default '0000' or '1234' remain common).

Bluetooth 2.1 — Secure Simple Pairing (SSP)

SSP was introduced to eliminate the usability burden of PIN entry while improving security. It uses **Elliptic Curve Diffie-Hellman (ECDH)** key exchange on the P-192 curve for forward secrecy. SSP has four association models:

- **Numeric Comparison:** Both devices display a 6-digit number; users confirm they match. Resistant to MITM.

- **Passkey Entry:** One device displays a 6-digit passkey; user enters it on the other. Resistant to MITM.
- **Just Works:** No user interaction. Convenient but provides *no MITM protection* — most IoT devices use this.
- **Out-of-Band (OOB):** Key exchange via NFC or another channel. Strongest protection.

BLE Security Manager Protocol (SMP) — LE Legacy Pairing and LE Secure Connections

BLE introduced its own pairing framework. **LE Legacy Pairing** (BT 4.x) generates a Short-Term Key (STK) from which a Long-Term Key (LTK) is derived — vulnerable to passive eavesdropping if the pairing exchange is captured. **LE Secure Connections** (BT 4.2+) uses ECDH on P-256, providing significantly stronger guarantees.

2.4 Bluetooth Security Modes

Security Mode	Applies To	Description
Mode 1 (Non-secure)	BT Classic 1.x/2.x	No security enforced. Device accepts all connections.
Mode 2 (Service-level)	BT Classic	Security enforced at L2CAP/RFCOMM service level after channel establishment.
Mode 3 (Link-level)	BT Classic	Security enforced at the link layer before channel establishment.
Mode 4 (SSP)	BT 2.1+	Mandatory for BT 2.1+. Uses SSP. Service-level enforcement with ECDH.
LE Security Mode 1	BLE	Levels 1–4: no security → unauthenticated → authenticated → authenticated+SC.
LE Security Mode 2	BLE	Data signing without encryption using CSRK.

Table 2.2 — Bluetooth Security Modes

2.5 Common Bluetooth Vulnerabilities

Vulnerability	Affected Versions	Description	CVE Examples
Bluejacking	All	Sending unsolicited messages via OPP to visible devices	N/A (social)
Bluesnarfing	BT 1.x-2.0	Unauthorised access to contacts, calendar data via OBEX push without auth	CVE-2003-0331

Bluebugging	BT 1.x-2.0	Full device control via AT commands via RFCOMM	CVE-2004-0229
BlueBorne	All unpatched	RCE via airborne vector; 8 vulnerabilities in BT stacks	CVE-2017-0781 to 0785
KNOB Attack	BT Classic unpatched	Entropy reduction in E0 key negotiation; brute-force decryption	CVE-2019-9506
BIAS Attack	BT Classic	Authentication bypass via role switch; impersonation without LTK	CVE-2020-10135
BLESA	BLE	Reconnection spoofing; device presents as bonded peer without re-authenticating	CVE-2020-9770
SweynTooth	BLE SoCs	13 flaws in BT 4.6-5.3 SDKs; crash, deadlock, security bypass	CVE-2019-17517+
BrakTooth	BLE Classic	22 flaws in commercial BT 5.2 SDKs; DoS and code execution	CVE-2021-28139+
MITM via SSP Just Works	BT 2.1+	Attacker relays Just Works pairing; no user confirmation required	N/A (design)

Table 2.3 — Notable Bluetooth Vulnerabilities

2.6 The Threat Landscape

Bluetooth security threats fall into several categories relevant to penetration testers:

- **Eavesdropping:** Passive capture of unencrypted or weakly encrypted Bluetooth traffic.
- **Device Impersonation:** Cloning a trusted device's identity to gain unauthorised access.
- **Service Exploitation:** Exploiting vulnerabilities in specific profiles (HID, OPP, RFCOMM).
- **Denial of Service:** Flooding a device with malformed packets or exhausting connection slots.
- **Relay / MITM Attacks:** Sitting between two communicating devices to intercept or modify data.
- **Firmware Attacks:** Exploiting vulnerabilities in Bluetooth controller firmware.

Chapter Summary

- ✓ BD_ADDR is a 48-bit address analogous to MAC; its OUI reveals device manufacturer.
- ✓ Class of Device (CoD) classifies device type and service capabilities — Spoonftooth can modify both.
- ✓ SSP (BT 2.1+) uses ECDH for forward secrecy; Just Works mode provides no MITM protection.
- ✓ BLE introduced distinct security modes; LE Secure Connections (BT 4.2+) significantly improves protection.
- ✓ The attack surface ranges from legacy PIN-based pairing to modern SoC firmware vulnerabilities.

CHAPTER 3

BlueZ: The Linux Bluetooth Stack

3.1 BlueZ Architecture Overview

BlueZ is the official Linux Bluetooth protocol stack, maintained as part of the Linux kernel and by the BlueZ project at bluez.org. It provides both in-kernel drivers and user-space daemons that together implement the full Bluetooth Classic and BLE stack on Linux systems, including Kali Linux.

BlueZ is architecturally split into kernel-space and user-space components, communicating via sockets and the D-Bus inter-process communication framework:

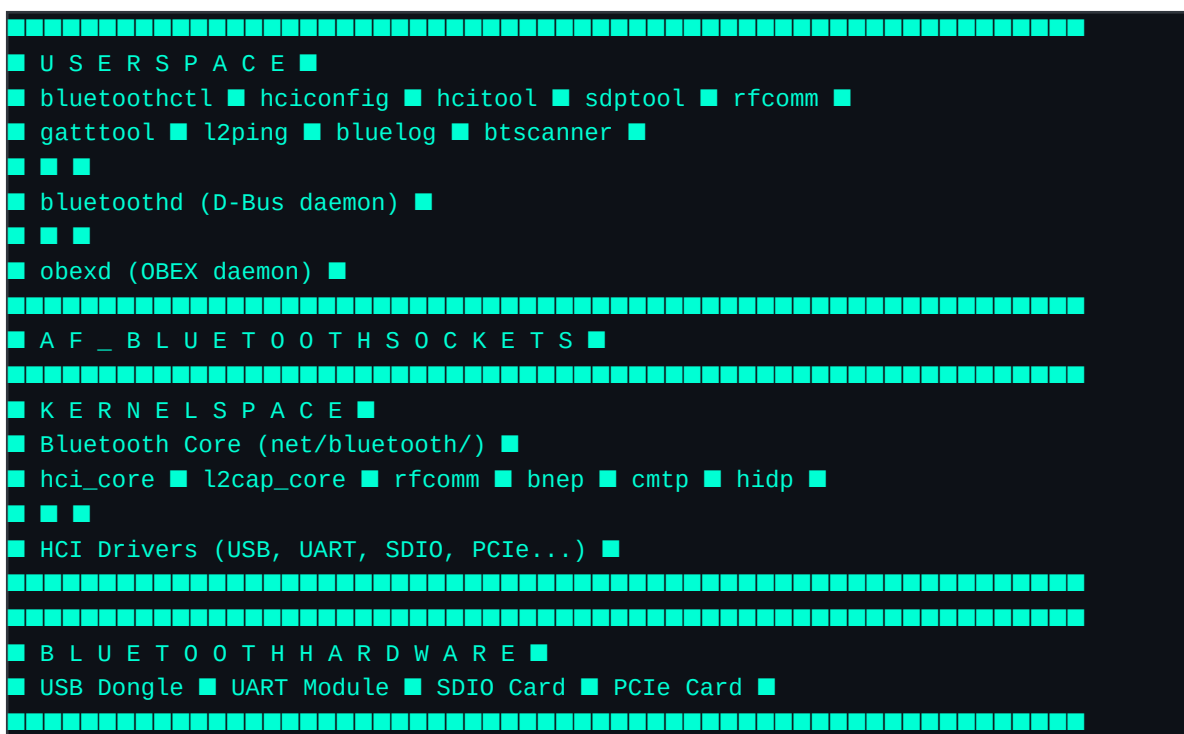


Figure 3.1 — BlueZ Architecture Diagram

3.2 Core Components and Daemons

The primary user-space daemon is **bluetoothd**, which exposes the BlueZ D-Bus API. Key components:

- **bluetoothd**: Central daemon. Manages adapters, devices, pairing, and profiles. Exposes org.bluez D-Bus interface.
- **obexd**: Handles OBEX file transfer. Exposes org.bluez.obex D-Bus interface.

- **bluetooth.service:** systemd unit managing bluetoothd lifecycle.
- **/etc/bluetooth/main.conf:** Primary bluetoothd configuration (discoverability, pairing policy, etc.).
- **/var/lib/bluetooth/:** Bonding database; contains BD_ADDR-named directories with paired device info.

3.3 HCI – Host Controller Interface

HCI is the standardised interface between the host stack and the controller hardware. In Linux, each Bluetooth adapter appears as an **hci** interface (hci0, hci1, etc.). The HCI command set allows the host to:

- Reset and configure the controller
- Read and write the local BD_ADDR and device name
- Initiate and manage inquiry/scanning procedures
- Establish and tear down ACL and SCO connections
- Configure link encryption and authentication parameters
- Set the Class of Device field

Reading HCI Device Information:

```
$ hciconfig -a
hci0: Type: Primary Bus: USB
BD Address: B8:27:EB:AA:BB:CC ACL MTU: 1021:8 SCO MTU: 64:1
UP RUNNING PSCAN ISCAN
RX bytes:1280 acl:0 sco:0 events:52 errors:0
TX bytes:3120 acl:0 sco:0 commands:51 errors:0
Features: 0xff 0xfe 0x0f 0xfe 0xdb 0xff 0x7b 0x87
Packet type: DM1 DM3 DM5 DH1 DH3 DH5 HV1 HV2 HV3
Link policy: RSWITCH SNIFF
Link mode: SLAVE ACCEPT
Name: 'kali-bt'
Class: 0x000000
Service Classes: Unspecified
Device Class: Miscellaneous,
HCI Version: 5.0 (0x9) Revision: 0x1
LMP Version: 5.0 (0x9) Subversion: 0x6119
Manufacturer: Realtek Semiconductor Corporation (93)
```

3.4 D-Bus API and bluetoothctl

The modern BlueZ API is exposed over D-Bus under the service name **org.bluez**. The command-line interface **bluetoothctl** is an interactive shell wrapping the D-Bus API. Key bluetoothctl commands:

Command	Description
---------	-------------

list	List available Bluetooth controllers
select	Select a specific controller
show [ctrl]	Show controller information
power on/off	Power adapter on or off
discoverable on/off	Toggle discoverability
pairable on/off	Toggle pairing acceptance
scan on/off	Start/stop device scan
devices	List discovered devices
info	Show detailed device information
pair	Initiate pairing with device
connect	Connect to a paired device
trust/untrust	Mark device as trusted/untrusted
remove	Remove bonding information
agent NoInputNoOutput	Register a Just Works agent

Table 3.1 — bluetoothctl Command Reference

3.5 Key BlueZ Configuration Files

/etc/bluetooth/main.conf

```
[Policy]
# Automatically enable Bluetooth on startup
AutoEnable=true

[General]
# Name displayed to remote devices
Name = MyLinuxDevice

# Discoverable mode: always on
DiscoverableTimeout = 0

# Class of Device (hex)
# 0x002540 = Smartphone
Class = 0x000000

# Allow all security modes
JustWorksRepairing = always
```

3.6 BlueZ Tools Reference

Tool	Package	Function	Security Use
hciconfig	bluez	Configure HCI devices	Read/set BD_ADDR, Class, Name
hcidtool	bluez	HCI-level commands	Inquiry, scan, RSSI, vendor commands
sdptool	bluez	SDP service browsing	Enumerate services on remote devices
l2ping	bluez	Ping via L2CAP	Test connectivity; DoS research
rfcomm	bluez	RFCOMM serial connections	Connect to serial services
btmon	bluez	HCI packet monitor	Real-time traffic analysis
gatttool	bluez	GATT attribute operations	BLE characteristic read/write
bluetoothctl	bluez	Interactive BT controller	Scan, pair, manage devices
btscanner	btscanner	Curses-based BT scanner	Visual discovery interface
bluelog	bluelog	Bluetooth logger	Passive device logging to file
spooftooth	spooftooth	BT identity spoofing	BD_ADDR/name/class manipulation

Table 3.2 — BlueZ and Kali Bluetooth Tools

Chapter Summary

- ✓ BlueZ provides both in-kernel drivers and user-space daemons for complete Bluetooth support on Linux.
- ✓ Each adapter appears as an hciN interface; HCI commands allow full configuration.
- ✓ bluetoothd exposes the D-Bus API; bluetoothctl wraps it interactively.
- ✓ Key config files: /etc/bluetooth/main.conf and /var/lib/bluetooth/ (bonding database).
- ✓ Security professionals use the full BlueZ toolchain; Spoonftooth integrates tightly with hciconfig at the HCI layer.

CHAPTER 4

Spooftooph: History, Background, and Design

4.1 Origins and Development History

Spooftooph was created by **JP Dunning (JP)**, a security researcher who recognised that while Bluetooth scanning and exploitation tools were available in the early 2000s, there was no dedicated utility for programmatically manipulating the local Bluetooth adapter's identity parameters. The tool was developed to address a specific gap in penetration testing workflows: the ability to impersonate authorised Bluetooth devices in order to test access controls, pairing policies, and trust relationships.

The project was developed as an open-source C program, originally hosted on the SourceForge platform and later mirrored in various security tool repositories. Its inclusion in the Kali Linux distribution (and before that, in BackTrack Linux) gave Spooftooph mainstream visibility within the penetration testing community.

Milestone	Description
~2007	Initial release; basic BD_ADDR spoofing via hciconfig
~2009	Added device name and Class of Device manipulation
~2011	Log file support for batch spoofing from scan results
2013+	Included in BackTrack Linux; wider distribution
2015+	Migrated to Kali Linux official repositories
2018+	Package updated for BlueZ 5.x compatibility
Present	Maintained in kali-linux-everything; available via apt

Table 4.1 — Spooftooph Development Timeline

4.2 Purpose and Design Philosophy

Spooftooph is a purpose-built tool with a narrow, well-defined scope: it modifies the three primary identity parameters of a local Bluetooth adapter:

1. **BD_ADDR (Bluetooth Device Address)** — the hardware address identifying the adapter.
2. **Device Name** — the human-readable name broadcast during inquiry and displayed in device lists.

3. Class of Device (CoD) — the numeric code indicating device type and service capabilities.

The tool is intentionally minimal. It does not provide scanning, exploitation, packet injection, or connection management capabilities — these are handled by other tools in the Kali Linux ecosystem. This design philosophy aligns with the Unix principle of doing one thing well.

4.3 How Spooftooph Modifies Bluetooth Identity

Understanding how Spooftooth achieves identity modification requires understanding how Bluetooth identity parameters are stored and set in the BlueZ stack:

BD ADDR Modification

The BD_ADDR is normally stored in the controller's firmware/EEPROM. Standard HCI commands do not provide a mechanism to change it — the HCI_Write_BD_ADDR command is not part of the Bluetooth specification. However, many controllers implement **vendor-specific HCI commands** that allow the host to override the BD_ADDR. Spooftooth leverages these vendor-specific commands (or, where supported, hciconfig's bdaddr extension) to write a new BD_ADDR to the controller.

NOTE

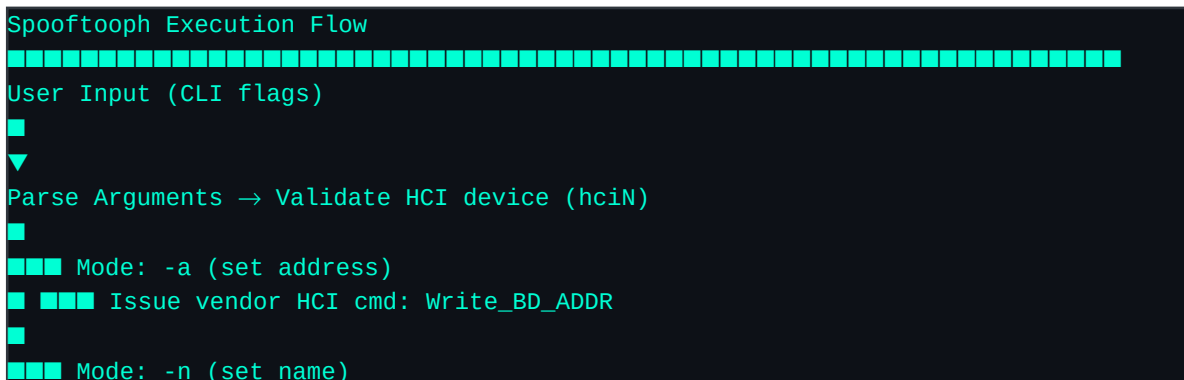
Not all Bluetooth controllers support vendor-specific BD_ADDR write commands. CSR (Cambridge Silicon Radio), Broadcom, and Realtek-based adapters are among the most commonly supported. Intel adapters may require different vendor commands. The 'bdaddr' utility from the bluez-hcidump package can independently verify support.

Device Name Modification

The device name is set via the standard HCI command **HCI_Write_Local_Name** (OGF=0x03, OCF=0x0013). This stores a UTF-8 string of up to 248 bytes in the controller. BlueZ wraps this in the `hciconfig` name command and the `D-Bus org.bluez.Adapter1.Name` property. Spooftooth calls this directly through the raw HCI socket interface.

Class of Device Modification

CoD is set via the standard HCI command **HCI_Write_Class_of_Device** (OGF=0x03, OCF=0x0024), which accepts a 3-byte value. SpoofTooph accepts this as a hexadecimal parameter and issues the HCI command directly.



[illegible]

Figure 4.1 — Spooftoph Internal Execution Flow

4.4 Spooftooph in the Kali Linux Ecosystem

SpoofTooth is categorised under Kali Linux's **wireless attacks** toolset and is included in the kali-linux-wireless and kali-linux-everything metapackages. It integrates naturally into Bluetooth assessment workflows alongside:

- **btscanner:** Visual discovery of nearby Bluetooth devices
- **bluelog:** Passive logging of discovered devices to a structured log file (compatible with SpoofTooth's -r flag)
- **hcitool/hciconfig:** Low-level adapter configuration and inquiry
- **sdptool:** Service enumeration after spoofed connection is established
- **Metasploit Framework:** Post-spoofing exploitation modules (l2cap_dos, obex_upload, etc.)

4.5 Limitations and Scope

Security professionals must understand Spooftooth's limitations before incorporating it into a testing workflow:

- **Hardware dependency:** BD_ADDR spoofing requires a controller with vendor-specific address write support. This is not universal.
- **Temporary changes:** Changes to BD_ADDR made via vendor commands are often lost on adapter reset or cold reboot. The name and CoD are persistent.
- **Not a traffic tool:** Spooftooth does not capture, inject, or replay Bluetooth packets. Use btmon or Ubertooth for traffic analysis.
- **BLE limitations:** BLE random address generation is independent of HCI vendor commands; Spooftooth primarily targets BR/EDR (Classic) adapters.
- **Single-adapter scope:** One invocation manages one adapter (specified via -i). Multi-adapter setups require separate invocations.

- **No persistence without scripting:** For persistent spoofing across reboots, the tester must incorporate SpoofTooph into init scripts or systemd units.

Chapter Summary

- ✓ SpoofTooph was created by JP Dunning to fill a gap in Bluetooth penetration testing tooling.
- ✓ It modifies three identity parameters: BD_ADDR, device name, and Class of Device.
- ✓ BD_ADDR changes rely on vendor-specific HCI commands — not all controllers support this.
- ✓ The tool is intentionally minimal: it modifies identity only, leaving scanning and exploitation to other tools.
- ✓ Hardware support and adapter reset behaviour are the primary limitations to account for in engagements.

CHAPTER 5

Installation and Environment Setup

5.1 Hardware Requirements

To use Spooftooph effectively for security testing, the following hardware is required or strongly recommended:

Component	Requirement	Recommended Options
Bluetooth Adapter (primary)	Mandatory	CSR8510-based dongle (Cambridge Silicon Radio) Realtek RTL8761B/RTL8852 USB dongle Broadcom BCM20702 USB dongle
Bluetooth Adapter (target)	Mandatory for labs	Second CSR dongle, Android device, Arduino HC-05/06
Processor	x86_64 / ARM64	Any modern laptop; Raspberry Pi 4 works well for portable labs
RAM	Minimum 2 GB	4 GB+ recommended for Kali VM
OS	Kali Linux 2022+	Kali Rolling; also Kali on Raspberry Pi
USB ports	2+ for dual-adapter	USB 2.0 or 3.0; use USB hub if needed

Table 5.1 — Hardware Requirements for Spooftooph Labs

NOTE

The CSR8510 A10 chipset (found in inexpensive USB dongles, commonly labelled 'Generic Bluetooth 4.0') is the most universally supported adapter for BD_ADDR spoofing. The 'bdaddr' utility can be used to pre-verify support: **bdaddr -i hci0**

5.2 Installing Spooftooph on Kali Linux

Kali Linux includes Spooftooph in its official repositories. Installation is straightforward:

Method 1: apt (Recommended)

```
# Update package index
$ sudo apt update

# Install spooftooph
$ sudo apt install spooftooph -y

# Verify installation
$ spooftooph -h
```

```
spooftooph v0.5.2
Usage: spooftooph -i [-a ] [-n ] [-d ]
[-s ] [-S ] [-r ]
[-l ]

# Check installed version
$ dpkg -l | grep spooftooph
ii spooftooph 0.5.2-1kali4 amd64 Automate spoofing or cloning BT devices
```

Method 2: Install from Kali Wireless Metapackage

```
# Install the full Kali wireless toolkit (includes spooftooph and all BT tools)
$ sudo apt install kali-linux-wireless -y

# This also installs: btscanner, bluelog, blueranger, redfang,
# hcitools, bluez-utils, rfkill, and many other wireless tools
```

Method 3: Build from Source

```
# Install build dependencies
$ sudo apt install build-essential libbluetooth-dev git -y

# Clone the source repository
$ git clone https://github.com/tw0phi/spooftooph.git
$ cd spooftooph

# Review the source
$ ls -la
total 48
-rw-r--r-- 1 kali kali 1095 Jan 10 2020 Makefile
-rw-r--r-- 1 kali kali 18432 Jan 10 2020 spooftooph.c
-rw-r--r-- 1 kali kali 1280 Jan 10 2020 README

# Compile
$ make
gcc -Wall -o spooftooph spooftooph.c -lbluetooth

# Install to system
$ sudo make install
install -m 755 spooftooph /usr/bin/

# Verify
$ which spooftooph
/usr/bin/spooftooph
```

5.3 Installing on Debian/Ubuntu Systems

```
# On Ubuntu 20.04/22.04 or Debian 11/12
```

```
# Install BlueZ development libraries
$ sudo apt install libbluetooth-dev build-essential -y

# Add the Kali Linux repository (optional; for pre-built package)
# WARNING: Only do this in isolated test VMs – mixing Kali repos
# with production Ubuntu/Debian systems can break dependencies.
$ echo 'deb http://http.kali.org/kali kali-rolling main' \
| sudo tee /etc/apt/sources.list.d/kali.list
$ wget -q -O - https://archive.kali.org/archive-key.asc \
| sudo apt-key add -
$ sudo apt update
$ sudo apt install spooftooph -y

# OR: build from source (safer for non-Kali Debian systems)
$ git clone https://github.com/tw0phi/spooftooph.git
$ cd spooftooph && make && sudo make install
```

5.4 Verifying Bluetooth Adapter Support

```
# Step 1: Identify your Bluetooth adapter
$ hciconfig -a

# Step 2: Check USB device details
$ lsusb | grep -i bluetooth
Bus 001 Device 003: ID 0a12:0001 Cambridge Silicon Radio, Ltd
Bluetooth Dongle (HCI mode)

# Step 3: Test BD_ADDR write capability with bdaddr utility
$ sudo apt install bluez-hcidump -y
$ sudo bdaddr -i hci0
Manufacturer: Cambridge Silicon Radio (10)
Device address: B8:27:EB:AA:BB:CC
# If no error → adapter supports BD_ADDR writing

# Step 4: Confirm kernel module
$ lsmod | grep bluetooth
bluetooth 700416 42 bnep,btbcm,btrtl,btintel,btusb,rfcomm

# Step 5: Check BlueZ daemon status
$ sudo systemctl status bluetooth
* bluetooth.service - Bluetooth service
Loaded: loaded (/lib/systemd/system/bluetooth.service; enabled)
Active: active (running) since ...
```

5.5 Configuring the Lab Environment

A properly isolated lab environment is essential for ethical Bluetooth testing. The following setup is recommended:

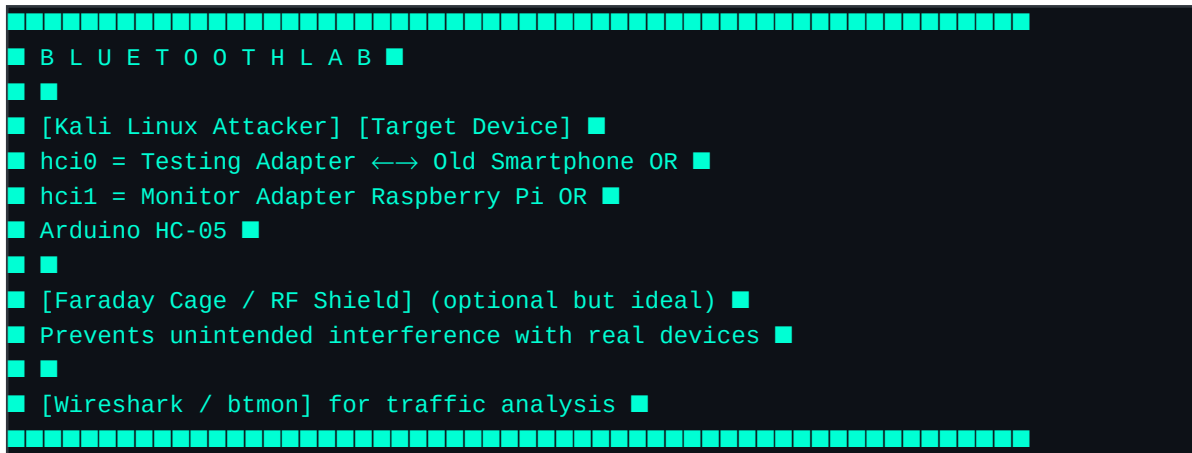


Figure 5.1 — Recommended Bluetooth Lab Topology

Lab Setup Commands:

```
# Ensure BlueZ is running
$ sudo systemctl start bluetooth
$ sudo systemctl enable bluetooth

# Bring up both adapters
$ sudo hciconfig hci0 up
$ sudo hciconfig hci1 up

# Make hci1 the 'victim' adapter (rename it)
$ sudo hciconfig hci1 name 'VictimDevice'
$ sudo hciconfig hci1 class 0x002540

# Verify both are visible
$ hciconfig
hci1: Type: Primary Bus: USB
BD Address: AA:BB:CC:DD:EE:FF ...
hci0: Type: Primary Bus: USB
BD Address: B8:27:EB:AA:BB:CC ...
```

5.6 Virtual Machine Considerations

When running Kali Linux in a virtual machine (VMware, VirtualBox, QEMU/KVM), Bluetooth USB adapters must be passed through to the guest. Native VM Bluetooth emulation is generally insufficient for security testing.

VMware Workstation / Fusion:

```
# Pass USB Bluetooth dongle to VM:
# VM → Settings → USB → Add USB Device → Select Bluetooth dongle
```

```
# In .vmx file (for automation):  
usb.generic.allowHID = "TRUE"  
usb.generic.allowLastHID = "TRUE"
```

VirtualBox:

```
# Install VirtualBox Extension Pack first (for USB 2.0/3.0 support)  
# Then: Settings → USB → Add USB Device Filter → Select Bluetooth dongle  
  
# Add your user to the 'vboxusers' group:  
$ sudo usermod -aG vboxusers $USER  
$ newgrp vboxusers  
  
# After passing through, in guest Kali:  
$ lsusb  
Bus 001 Device 002: ID 0a12:0001 Cambridge Silicon Radio, Ltd ...
```

Chapter Summary

- ✓ CSR8510-based USB dongles are the most widely supported for BD_ADDR spoofing.
- ✓ Install via apt on Kali Linux; build from source on other Debian systems.
- ✓ Verify adapter support with 'bdaddr -i hciX' before attempting spoofing.
- ✓ Use a dual-adapter setup for realistic lab testing: one 'attacker', one 'victim'.
- ✓ In VMs, always pass USB adapters through rather than relying on VM Bluetooth emulation.

CHAPTER 6

Complete Command Reference

6.1 Spoonftooth Syntax Overview

```
USAGE:
spoonftooth -i [OPTIONS]

MANDATORY:
-i Specify HCI interface (e.g., hci0, hci1)

SPOOFING OPTIONS (use one or more):
-a Set Bluetooth device address (format: AA:BB:CC:DD:EE:FF)
-n Set Bluetooth device name (max 248 chars)
-d Set Class of Device (hex, e.g., 0x002540)

SCAN/CLONE OPTIONS:
-s Scan for seconds; randomly clone a discovered device
-S Specify HCI device for scanning (default: same as -i)

LOG FILE OPTIONS:
-l Save scan results to a log file
-r Read device info from a log file and spoof randomly

HELP:
-h Print help and exit
```

6.2 All Flags and Options Explained

Flag	Argument	Type	Description	Example
-i		Required	HCI device for identity modification. Must be powered on.	spoonftooth -i hci0
-a		Optional	Target Bluetooth address in colon-separated hex format. Must be 6 octets.	spoonftooth -i hci0 -a AA:BB:CC:11:22:33
-n		Optional	Device name string. Wrap in quotes if it contains spaces. Max 248 bytes UTF-8.	spoonftooth -i hci0 -n 'My-Keyboard'
-d		Optional	Class of Device as 3-byte hex. May include or omit '0x' prefix.	spoonftooth -i hci0 -d 0x002540
-s		Optional	Perform active inquiry for this many seconds, then clone a random result.	spoonftooth -i hci0 -s 15

-S		Optional	Use a different HCI device for the inquiry scan (-s mode). Useful when primary adapter needs to be offline for address change.	spooftooph -i hci0 -s 20 -S hci1
-l		Optional	After scanning, write discovered device info to this file for later use with -r.	spooftooph -i hci0 -s 20 -l /tmp/bt_scan.log
-r		Optional	Read previously saved scan log; select a random device and spoof its parameters.	spooftooph -i hci0 -r /tmp/bt_scan.log
-h	none	Optional	Print usage information and exit.	spooftooph -h

Table 6.1 — Complete Spooftooph Flag Reference

6.3 Spoofing by Device Address (-a)

The -a flag directly sets the BD_ADDR of the local HCI adapter. This is the most fundamental operation and underpins all impersonation scenarios.

Basic Address Spoofing:

```
# Verify current address
$ hciconfig hci0 | grep 'BD Address'
BD Address: B8:27:EB:AA:BB:CC

# Spoof to a new address
$ sudo spooftooph -i hci0 -a 00:11:22:33:44:55
[+] Changing BD_ADDR of hci0 to 00:11:22:33:44:55
[+] Bringing hci0 down...
[+] Writing new BD_ADDR...
[+] Bringing hci0 up...
[+] BD_ADDR successfully changed!

# Verify change
$ hciconfig hci0 | grep 'BD Address'
BD Address: 00:11:22:33:44:55
```

Impersonating a Specific Device:

```
# First, discover the target device's address
$ sudo hcitool scan
Scanning ...
B8:27:EB:12:34:56 Raspberry-Pi-AP
00:1A:7D:DA:71:13 Samsung-TV
E4:17:D8:AC:BD:CE Apple-Watch

# Impersonate the Raspberry Pi
$ sudo spooftooph -i hci0 -a B8:27:EB:12:34:56

# Now set matching name and class
```

```
$ sudo spooftooph -i hci0 -n 'Raspberry-Pi-AP' -d 0x000104
```

6.4 Spoofing by Device Name (-n)

```
# Set device name only
$ sudo spooftooph -i hci0 -n 'iPhone 15'
[+] Setting device name on hci0 to: iPhone 15
[+] Done.

# Verify
$ hciconfig hci0 name
hci0: Name: 'iPhone 15'

# Use a name likely to be accepted by target access controls
$ sudo spooftooph -i hci0 -n 'Admin-Keyboard'

# Name with special characters (use quotes)
$ sudo spooftooph -i hci0 -n 'BT_Printer_HP_3210'
```

6.5 Spoofing by Device Class (-d)

The CoD value determines how a device is categorised. Common values for impersonation scenarios:

Device Type	CoD (Hex)	Description
Generic Smartphone	0x002540	Phone + Telephony + Object Transfer
iOS iPhone	0x00020C	Phone, Uncategorised, cellular
Android Phone	0x002508	Phone, Smart + Networking
Laptop/Notebook	0x000104	Computer, Laptop
Desktop PC	0x000100	Computer, Desktop
Bluetooth Speaker	0x200404	Audio/Video, Loudspeaker
Wireless Headset	0x200404	Audio/Video, Wearable headset
Keyboard	0x000540	Peripheral, Keyboard
Mouse	0x000580	Peripheral, Pointing
Printer	0x000680	Imaging, Printer
Smartwatch/Wearable	0x000704	Wearable, Wristwatch
Car Kit / Handsfree	0x002408	Phone, Car

Medical Glucose Sensor	0x000906	Health, Glucose Meter
Uncategorised	0x000000	Miscellaneous

Table 6.2 — Common CoD Values for Impersonation

```
# Spoof as a generic smartphone
$ sudo spooftooph -i hci0 -d 0x002540

# Verify class
$ hciconfig hci0 class
hci0: Class: 0x002540
Service Classes: Networking, Object Transfer
Device Class: Phone, Cellular
```

6.6 Scanning and Cloning (-s, -S)

The `-s` flag automates the full workflow: scan, pick a random discovered device, and clone its BD_ADDR, name, and CoD in a single command.

```
# Scan for 20 seconds, then clone a random device
$ sudo spooftooph -i hci0 -s 20
[+] Scanning for 20 seconds on hci0...
[+] Found: 00:1A:7D:DA:71:13 Samsung-TV 0x200404
[+] Found: B8:27:EB:12:34:56 Pi-Hotspot 0x000104
[+] Found: E4:17:D8:AC:BD:CE Galaxy-S24 0x002540
[+] Randomly selected: Galaxy-S24 (E4:17:D8:AC:BD:CE, 0x002540)
[+] Applying BD_ADDR: E4:17:D8:AC:BD:CE
[+] Applying Name: Galaxy-S24
[+] Applying CoD: 0x002540
[+] Clone complete. hci0 now impersonates: Galaxy-S24

# Use a separate adapter for scanning (hci1) while modifying hci0
# (Useful when hci0 must be taken offline for address change)
$ sudo spooftooph -i hci0 -s 30 -S hci1
```

6.7 Log File Operations (-l, -r)

```
# Scan and save results to log file
$ sudo spooftooph -i hci0 -s 30 -l /tmp/bt_devices.log

# Inspect the log file
$ cat /tmp/bt_devices.log
00:1A:7D:DA:71:13|Samsung-TV|0x200404
B8:27:EB:12:34:56|Pi-Hotspot|0x000104
E4:17:D8:AC:BD:CE|Galaxy-S24|0x002540
A4:C3:F0:11:22:33|HP-Printer|0x000680

# Spoof from an existing log (no new scan needed)
```

```
$ sudo spooftooph -i hci0 -r /tmp/bt_devices.log
[+] Read 4 devices from /tmp/bt_devices.log
[+] Randomly selected: HP-Printer (A4:C3:F0:11:22:33, 0x000680)
[+] Applying identity...
[+] Done.

# The log format is pipe-separated: ADDRESS|NAME|CLASS
# You can manually edit it to add entries
$ echo 'DE:AD:BE:EF:CA:FE|TestDevice|0x002540' >> /tmp/bt_devices.log
```

6.8 Specifying HCI Interface (-i)

```
# List available interfaces
$ hciconfig
hci0: Type: Primary Bus: USB BD Address: B8:27:EB:AA:BB:CC
hci1: Type: Primary Bus: USB BD Address: 00:1A:7D:DA:71:13

# Target hci0
$ sudo spooftooph -i hci0 -a DE:AD:BE:EF:CA:FE

# Target hci1
$ sudo spooftooph -i hci1 -n 'TestAdapter'

# Reset adapter to original address (requires a power cycle or reboot
# for some adapters; for CSR adapters, bdaddr tool can restore it)
$ sudo hciconfig hci0 down
$ sudo hciconfig hci0 up
$ hciconfig hci0 | grep BD
# If the original address restores on power cycle, you'll see it here
```

6.9 Combined Usage Patterns

Full Identity Clone (manual):

```
$ sudo spooftooph -i hci0 \
-a DE:AD:BE:EF:CA:FE \
-n 'Target-Device-Name' \
-d 0x002540
```

Scan, Log, and Clone in Stages:

```
# Stage 1: Passive reconnaissance (use hci1 for scanning, don't modify hci0)
$ sudo spooftooph -i hci0 -s 60 -S hci1 -l /tmp/targets.log

# Stage 2: Review log; choose a specific target by editing the log
$ cat /tmp/targets.log
$ nano /tmp/targets.log # Remove all but the desired target
```

```
# Stage 3: Apply spoofed identity
$ sudo spoonftooth -i hci0 -r /tmp/targets.log
```

Periodic Re-Cloning (Shell Loop):

```
#!/bin/bash
# Continuously re-clone a random device from saved log every 60 seconds
while true; do
sudo spoonftooth -i hci0 -r /tmp/targets.log
echo '[*] Re-cloned. Sleeping 60s...'
sleep 60
done
```

Chapter Summary

- ✓ Spoonftooth's core flags are: -i (adapter), -a (address), -n (name), -d (CoD).
- ✓ The -s flag automates scan-and-clone; -S lets a separate adapter handle scanning.
- ✓ Log files (-l to write, -r to read) enable staged reconnaissance and batch spoofing.
- ✓ All three identity parameters can be changed in a single command invocation.
- ✓ For periodic or persistent spoofing, wrap Spoonftooth in a shell loop or systemd service.

CHAPTER 7

Bluetooth Reconnaissance Techniques

7.1 Passive vs. Active Reconnaissance

Bluetooth reconnaissance can be passive or active. The distinction matters both legally (passive observation versus active interrogation) and technically (passive cannot discover non-discoverable devices; active inquiry reveals more information but generates detectable traffic):

Aspect	Passive Reconnaissance	Active Reconnaissance
Traffic generated	None from attacker	Inquiry requests sent; creates HCI events
Detectability	Invisible to targets	May appear in device logs and IDS
Discoverability	Only advertising devices	All devices in range that respond to inquiries
Tools	Ubertooth One, bluelog	hcitool, btscanner, spoonftooth -s
Use case	Mapping environment; deniability	Targeted assessment; faster results
Legal risk	Lower (observation only)	Higher (active interrogation)

Table 7.1 — Passive vs. Active Bluetooth Reconnaissance

7.2 Device Discovery with hcitool

hcitool is the primary command-line interface for Bluetooth inquiry operations. It communicates directly with the HCI layer:

```
# Standard inquiry (2.56 seconds, ~1 page of results)
$ sudo hcitool scan
Scanning ...
AA:BB:CC:DD:EE:FF Galaxy-S24
B8:27:EB:12:34:56 Raspberry-Pi

# Extended inquiry with RSSI (signal strength) and CoD
$ sudo hcitool inq --rssi
Inquiring ...
AA:BB:CC:DD:EE:FF clock offset: 0x0a3c class: 0x002540 rssi: -65
B8:27:EB:12:34:56 clock offset: 0x1d2a class: 0x000104 rssi: -78

# Look up device name only
$ sudo hcitool name AA:BB:CC:DD:EE:FF
Galaxy-S24
```

```
# List paired/connected devices
$ sudo hcitool con
Connections:
< ACL AA:BB:CC:DD:EE:FF handle 1 state 1 lm MASTER

# Get detailed info on a connected device
$ sudo hcitool info AA:BB:CC:DD:EE:FF
Requesting information ...
BD Address: AA:BB:CC:DD:EE:FF
Device Name: Galaxy-S24
LMP Version: 5.1 (0xa) LMP Subversion: 0x1234
Manufacturer: Qualcomm Technologies International, Ltd. (29)
Features: ...
```

7.3 Service Enumeration with sdptool

sdptool queries a device's Service Discovery Protocol (SDP) server to reveal all active profiles and services — an invaluable step in understanding a target's attack surface:

```
# Browse all services on a target device
$ sudo sdptool browse AA:BB:CC:DD:EE:FF
Browsing AA:BB:CC:DD:EE:FF ...
Service Name: Headset Gateway
Service RecHandle: 0x10000
Service Class ID List:
'Headset Audio Gateway' (0x1112)
'Generic Audio' (0x1203)
Protocol Descriptor List:
'L2CAP' (0x0100)
'RFCOMM' (0x0003)
Channel: 1

Service Name: Handsfree Gateway
...
Service Name: AV Remote Control Target
...

# Search for a specific service
$ sudo sdptool search --bdaddr AA:BB:CC:DD:EE:FF OPUSH
$ sudo sdptool search --bdaddr AA:BB:CC:DD:EE:FF RFCOMM

# Add a service record locally (for testing responses)
$ sudo sdptool add --channel=3 SP
Serial Port service registered
```

7.4 Scanning with btscanner and Bluelog

btscanner provides a full-screen ncurses interface for visual device monitoring:

```
$ sudo apt install btscanner -y
$ sudo btscanner

# Navigation:
# i = start inquiry n = name lookup
# q = quit arrow keys = navigate
# btscanner shows: BD_ADDR, name, class, RSSI, last seen
```

Bluelog passively logs discovered devices to a file compatible with Spooftooth's -r flag:

```
$ sudo apt install bluelog -y

# Log discovered devices (run for 5 minutes)
$ sudo bluelog -i hci0 -o /tmp/bluelog.log -t 300 -v

# The output format is compatible with spooftooph -r:
$ cat /tmp/bluelog.log
00:1A:7D:DA:71:13|Samsung-TV|0x200404
B8:27:EB:12:34:56|Pi-Hotspot|0x000104

# Feed directly into spooftooph
$ sudo spooftooph -i hci0 -r /tmp/bluelog.log
```

7.5 Mapping the Bluetooth Environment

Building a complete map of a Bluetooth environment requires combining multiple tools and recording results systematically. A structured approach:

```
Bluetooth Reconnaissance Workflow
```

```
PHASE 1: PASSIVE LOGGING  
■■■ sudo bluelog -i hci1 -o scan.log -t 600  
Quietly logs all advertising devices for 10 mins  
  
PHASE 2: ACTIVE INQUIRY  
■■■ sudo hcitool inq --rssi > inquiry.txt  
Gets BD_ADDR, CoD, RSSI of all discoverable devices  
  
PHASE 3: NAME RESOLUTION  
■■■ for addr in $(awk '{print $1}' inquiry.txt); do  
sudo hcitool name $addr; done  
  
PHASE 4: SERVICE ENUMERATION  
■■■ sudo sdptool browse  
Reveals profiles: RFCOMM channels, OPP, HFP, A2DP  
  
PHASE 5: DOCUMENTATION  
■■■ Compile: Address | Vendor (OUI lookup) | Name |
```



Figure 7.1 — Structured Bluetooth Reconnaissance Workflow

7.6 Information Gathering Workflow

The following table summarises what each piece of discovered information reveals and how it feeds into subsequent testing phases:

Data Point	How Gathered	What It Reveals	Use in Testing
BD_ADDR	hcitool scan	Vendor (OUI); uniquely identifies device	Target for spoofing; OUI reveals manufacturer
Device Name	hcitool name	User-assigned name; may reveal model/role	Enables convincing impersonation
Class of Device	hcitool inq	Device type and services	Profile fingerprinting; spoof with -d
RSSI	hcitool inq --rssi	Relative proximity	Prioritise targets; physical location
SDP Records	sdptool browse	Active profiles and RFCOMM channels	Identify exploitable services
LMP Version	hcitool info	Bluetooth version and features	Identify legacy vulnerabilities
Manufacturer	hcitool info	Chipset vendor	Known firmware vulnerabilities
Inquiry Mode	Discoverable/ISCAN	Device discoverability policy	Indicates security awareness

Table 7.2 — Bluetooth Intelligence Data Points

Chapter Summary

- ✓ Passive reconnaissance generates no traffic; active inquiry reveals more information but may be detected.
- ✓ hcitool scan, inq, name, and info form the core active reconnaissance toolkit.
- ✓ sdptool browse reveals all SDP services — the key attack surface enumeration step.
- ✓ btscanner and Bluelog provide visual and logged discovery; Bluelog output is Spoonftooth-compatible.
- ✓ Systematic documentation of BD_ADDR, Name, CoD, RSSI, and SDP records forms the reconnaissance deliverable.

CHAPTER 8

Practical Laboratory Exercises

■ WARNING

ALL laboratory exercises in this chapter must be performed ONLY in an isolated, controlled lab environment using devices you own or have explicit written permission to test. Never perform these exercises in public areas or on devices belonging to others. Bluetooth signals travel through walls — use RF shielding where possible.

8.1 Lab 1: Setting Up the Test Environment

Objective: Configure a dual-adapter Kali Linux environment for Bluetooth testing.

Equipment: Kali Linux system, 2× CSR8510 USB Bluetooth adaptors.

Steps:

```
# 1. Plug in both USB adapters and verify detection
$ lsusb | grep -i bluetooth
Bus 001 Device 003: ID 0a12:0001 Cambridge Silicon Radio, Ltd BT Dongle
Bus 001 Device 004: ID 0a12:0001 Cambridge Silicon Radio, Ltd BT Dongle

# 2. Load Bluetooth modules if not auto-loaded
$ sudo modprobe btusb
$ sudo modprobe bluetooth

# 3. Start BlueZ
$ sudo systemctl start bluetooth

# 4. Bring both interfaces up
$ sudo hciconfig hci0 up
$ sudo hciconfig hci1 up

# 5. Verify both are active
$ hciconfig
hci0: Type: Primary Bus: USB BD Address: B8:27:EB:AA:BB:CC
UP RUNNING PSCAN ISCAN
hci1: Type: Primary Bus: USB BD Address: 00:1A:7D:DA:71:13
UP RUNNING PSCAN ISCAN

# 6. Make hci1 discoverable (it will be our 'victim' device)
$ sudo hciconfig hci1 piscan
$ sudo hciconfig hci1 name 'TestVictimDevice'
$ sudo hciconfig hci1 class 0x002540
```

```
# Expected result: hci1 is now discoverable as 'TestVictimDevice'
# with a smartphone class
```

Expected Outcome: Two HCI adapters active; hci1 broadcasting as a named, discoverable device.

8.2 Lab 2: Basic Device Discovery

Objective: Discover and document Bluetooth devices using hcitool and sdptool.

```
# From hci0 (our 'attacker' adapter), scan for nearby devices
$ sudo hcitool scan
Scanning ...
00:1A:7D:DA:71:13 TestVictimDevice

# Extended inquiry with RSSI and CoD
$ sudo hcitool inq --rssi
Inquiring ...
00:1A:7D:DA:71:13 clock offset: 0x0001 class: 0x002540 rssi: -45

# Resolve device name
$ sudo hcitool name 00:1A:7D:DA:71:13
TestVictimDevice

# Enumerate services
$ sudo sdptool browse 00:1A:7D:DA:71:13
Browsing 00:1A:7D:DA:71:13 ...
# (Output depends on what services hci1 is running)

# Record results in a file for reference
$ echo '00:1A:7D:DA:71:13|TestVictimDevice|0x002540' > /tmp/victim.log
```

8.3 Lab 3: Cloning a Bluetooth Identity

Objective: Clone hci1's identity onto hci0 using Spooftooph.

```
# Record hci0's original identity
$ hciconfig hci0 | grep -E 'BD Address|Name|Class'
BD Address: B8:27:EB:AA:BB:CC
Name: 'kali'
Class: 0x000000

# Spoof hci0 to match hci1
$ sudo spooftooph -i hci0 \
-a 00:1A:7D:DA:71:13 \
-n 'TestVictimDevice' \
-d 0x002540
```

```
# Verify hci0 now has hci1's identity
$ hciconfig hci0 | grep -E 'BD Address|Name|Class'
BD Address: 00:1A:7D:DA:71:13
Name: 'TestVictimDevice'
Class: 0x002540

# Both hci0 and hci1 now advertise identical identities.
# A third device scanning would see two 'TestVictimDevice' entries
# with the same address – this demonstrates how a client could be
# confused about which is the legitimate device.

# Restore hci0 original identity when done
$ sudo hciconfig hci0 down && sudo hciconfig hci0 up
$ sudo hciconfig hci0 name 'kali'
$ sudo hciconfig hci0 class 0x000000
```

8.4 Lab 4: Spoofing Device Class

Objective: Change device class and observe how other devices categorise the adapter.

```
# Present hci0 as a Bluetooth printer
$ sudo spooftooph -i hci0 -d 0x000680

# Verify
$ hciconfig hci0 class
hci0: Class: 0x000680
Service Classes: Unspecified
Device Class: Imaging, Printer

# Present as wireless headset
$ sudo spooftooph -i hci0 -d 0x200404
$ hciconfig hci0 class
hci0: Class: 0x200404
Device Class: Audio/Video, Wearable Headset Device

# Present as medical health device
$ sudo spooftooph -i hci0 -d 0x000906
$ hciconfig hci0 class
hci0: Class: 0x000906
Device Class: Health, Blood Pressure Monitor
```

8.5 Lab 5: Log-File-Based Spoofing

Objective: Use scan log files to enable staged, repeatable spoofing operations.

```
# Create a sample log file with multiple devices
$ cat > /tmp/testdevices.log << 'EOF'
00:1A:7D:DA:71:13|TestVictimDevice|0x002540
```

```
B8:27:EB:12:34:56|RaspberryPi-Lab|0x000104
DE:AD:BE:EF:00:01|TestPrinter|0x000680
EOF

# Use -r to read from log and apply a random entry
$ sudo spooftooph -i hci0 -r /tmp/testdevices.log
[+] Read 3 devices from /tmp/testdevices.log
[+] Randomly selected: RaspberryPi-Lab (B8:27:EB:12:34:56, 0x000104)
[+] Applying identity...
[+] Done.

# Verify
$ hciconfig hci0 | grep -E 'BD Address|Name|Class'
BD Address: B8:27:EB:12:34:56
Name: 'RaspberryPi-Lab'
Class: 0x000104
```

8.6 Lab 6: Automated Scan-and-Clone

```
# Use spooftooph to scan with hci1 and apply to hci0
$ sudo spooftooph -i hci0 -s 20 -S hci1 -l /tmp/scan_results.log
[+] Scanning for 20 seconds on hci1...
[+] Found: 00:1A:7D:DA:71:13 TestVictimDevice 0x002540
[+] Saved to /tmp/scan_results.log
[+] Randomly selected: TestVictimDevice
[+] Applying to hci0: BD_ADDR=00:1A:7D:DA:71:13, Name=TestVictimDevice,
CoD=0x002540
[+] Done.
```

8.7 Lab 7: Bluetooth Impersonation Assessment

Objective: Demonstrate how a spoofed device identity could affect access controls in an authorised test environment.

This lab requires an authorised target device (e.g., a smartphone set to auto-connect to known devices) and assesses whether it connects to a clone of a known device without additional authentication:

```
# Context: Phone X is paired with 'TrustedSpeaker' (BD_ADDR: AA:BB:CC:DD:EE:FF)
# Hypothesis: If we clone TrustedSpeaker's identity, will the phone connect?

# Step 1: Document the target's full identity (obtained during recon)
TARGET_ADDR='AA:BB:CC:DD:EE:FF'
TARGET_NAME='TrustedSpeaker'
TARGET_COD='0x200404'

# Step 2: Apply spoofed identity to hci0
```

```
$ sudo spooftooph -i hci0 \  
-a $TARGET_ADDR \  
-n $TARGET_NAME \  
-d $TARGET_COD  
  
# Step 3: Make hci0 connectable  
$ sudo hciconfig hci0 piscan  
  
# Step 4: Monitor for connection attempts  
$ sudo btmon 2>&1 | grep -E 'connect|Connection'  
  
# Expected finding: If the phone attempts to connect, this  
# demonstrates that BD_ADDR alone is insufficient for authentication.  
# (Actual authentication would still require the stored link key.)  
  
# IMPORTANT: This test is only valid in an authorised lab environment.  
# Document findings; recommend link key enforcement and user notification.
```

8.8 Lab 8: Capture and Analysis with btmon

```
# btmon captures all HCI traffic on the host  
$ sudo btmon  
Bluetooth monitor ver 5.72  
= New Index: hci0 (Primary,USB,B8:27:EB:AA:BB:CC) [hci0] 1.001  
= Open Index: hci0 [hci0] 1.002  
  
# In a second terminal, run spooftooph while btmon captures  
$ sudo spooftooph -i hci0 -a DE:AD:BE:EF:CA:FE  
  
# btmon will show the HCI commands issued:  
< HCI Command: Reset (0x03|0x0003) [hci0] 1.003  
> HCI Event: Command Complete (0x0e) ncmd 1 [hci0] 1.004  
...  
< HCI Command: Write Local Name (0x03|0x0013) [hci0] 1.010  
Name: TestDevice  
< HCI Command: Write Class of Device (0x03|0x0024) [hci0] 1.011  
Class: 0x000000 (0x000000)  
  
# Save btmon output to file for analysis  
$ sudo btmon -w /tmp/capture.btsnoop  
  
# Open in Wireshark for deeper analysis  
$ wireshark /tmp/capture.btsnoop
```

Chapter Summary

- ✓ All labs require an isolated environment with written authorisation.
- ✓ Lab 1-2 establish the environment and reconnaissance baseline.

- ✓ Labs 3-6 demonstrate Spoonftooph's core identity manipulation capabilities.
- ✓ Lab 7 tests impersonation effectiveness in an access-control context.
- ✓ Lab 8 uses btmon to capture and inspect the underlying HCI command flow.

CHAPTER 9

Tool Comparison and Assessment Ecosystem

Spooftooph occupies a specific niche in the Bluetooth security toolkit. Understanding how it compares to adjacent tools helps practitioners choose the right tool for each phase of an assessment.

9.1 Spooftooph vs. hciconfig

Capability	Spooftooph	hciconfig
Set device name	Yes (-n)	Yes (hciconfig hci0 name 'X')
Set Class of Device	Yes (-d)	Yes (hciconfig hci0 class 0XXXXXXXX)
Set BD_ADDR	Yes (-a, vendor cmds)	Limited (requires bdaddr tool)
Scan and auto-clone	Yes (-s, -S)	No
Log file support	Yes (-l, -r)	No
GUI/interactive	No (CLI only)	No (CLI only)
Scripting-friendly	Yes	Yes
Part of BlueZ core	No (separate package)	Yes (bluez package)

Table 9.1 — Spooftooph vs. hciconfig

9.2 Spooftooph vs. BTLEJuice / GATTacker

GATTacker (and its successor BTLEJuice) focuses on **BLE MITM attacks** at the GATT attribute layer. It operates at a higher protocol level than Spooftooph:

Aspect	Spooftooph	GATTacker / BTLEJuice
Protocol target	Bluetooth Classic (BR/EDR)	Bluetooth Low Energy (BLE)
Attack type	Identity spoofing (layer 2)	GATT MITM (layer 7)
Connection handling	None (identity only)	Full connection relay
Data interception	No	Yes (reads/writes intercepted)

Characteristic modification	No	Yes
Use case	Impersonation testing	BLE data interception/modification

Table 9.2 — Spoonftooth vs. GATTacker

9.3 Spoonftooth vs. Bettercap (BLE Module)

Bettercap is a comprehensive network attack tool with an active BLE module. It provides scanning, characteristic enumeration, and data injection capabilities for BLE devices:

Capability	Spoonftooth	Bettercap (BLE module)
BT Classic support	Primary target	Limited
BLE support	Partial (address only)	Full GATT operations
Web UI	No	Yes (REST API + web interface)
Scripting	Shell scripts	JavaScript (caplets)
Active development	Maintenance mode	Actively developed
Learning curve	Low (simple CLI)	Moderate (caplet scripting)

Table 9.3 — Spoonftooth vs. Bettercap BLE

9.4 Overall Tool Selection Matrix

Scenario	Recommended Tool(s)
BD_ADDR / name / CoD spoofing (BT Classic)	Spoonftooth
Passive BT device logging	Bluelog
Visual BT discovery	btscanner
Service enumeration (SDP)	sdptool
L2CAP / HCI raw operations	hciconfig, hcitool, l2ping
BLE GATT read/write	gatttool, gattacker, bettercap
BLE MITM relay	GATTacker / BTLEJuice
Bluetooth traffic capture	btmon, Ubertooth One + Wireshark
Full wireless assessment suite	bettercap (with wireless.recon.show)
Post-exploitation / exploitation	Metasploit (BT modules)

Table 9.4 — Bluetooth Tool Selection Matrix

Chapter Summary

- ✓ SpoofTooph is uniquely focused on BT Classic identity manipulation — no other tool combines scan-and-clone with log file support.
- ✓ hciconfig can set name and CoD individually but lacks SpoofTooph's automation and BD_ADDR capability.
- ✓ GATTacker and Bettercap target BLE at the GATT layer — complementary to SpoofTooph, not competitive.
- ✓ Build a toolkit: Bluelog → SpoofTooph → sdptool → Metasploit covers a full BT Classic assessment pipeline.

CHAPTER 10

Legal and Ethical Framework

■ WARNING

This chapter is not legal advice. Laws vary by jurisdiction and change over time. Always consult a qualified legal professional before conducting security testing. Ignorance of the law is not a defence.

10.1 Applicable Laws and Regulations

Jurisdiction	Relevant Law	Key Provisions
India	IT Act 2000, Section 66	Unauthorised computer system access; imprisonment up to 3 years, fine up to ■5 lakh
India	IT (Amendment) Act 2008, Section 43	Damage to computer/system; civil liability without limit
USA	CFAA (18 U.S.C. § 1030)	Unauthorised access; felony for intentional damage
UK	Computer Misuse Act 1990	Unauthorised access; up to 10 years for supply of hacking tools
EU	Directive 2013/40/EU	Harmonised cybercrime laws; unauthorised access offence
Australia	Criminal Code Act 1995, Div 477	Unauthorised access; up to 10 years
All	GDPR / Privacy laws	Processing personal data (incl. BD_ADDR/location) without consent

Table 10.1 — Key Cybercrime Legislation by Jurisdiction

10.2 Written Authorisation and Scope

The cornerstone of legal penetration testing is a signed, written authorisation document — commonly called a **Rules of Engagement (RoE)** or **Letter of Authorisation (LoA)**. This document must specify:

1. The name and contact information of the authorising party (must have legal authority over the environment)
2. The precise scope: which devices, addresses, or MAC ranges are in-scope
3. The testing period: specific dates and times
4. Permitted techniques and tools (explicitly include Spoonftooth if applicable)
5. Out-of-scope items (devices, addresses, personnel)

6. Emergency contact and abort procedures
7. Data handling requirements for captured information
8. Signatures of both the client representative and the tester

■ NOTE

Even with authorisation, Bluetooth signals propagate beyond the intended test area. Always use a Faraday cage or RF shielding for lab exercises to prevent unintended interference with devices outside the scope of your engagement.

10.3 Responsible Disclosure

If your authorised testing reveals previously unknown vulnerabilities in vendor products, responsible disclosure ethics require:

1. Privately notify the vendor's security team (search for security@vendor.com or the vendor's PSIRT).
2. Provide a clear, reproducible description of the vulnerability with supporting evidence.
3. Allow a reasonable remediation period — typically 90 days per industry standard (Google Project Zero, CERT/CC).
4. Coordinate disclosure timing with the vendor where possible.
5. Publish technical details only after the vendor has released a patch or the deadline has passed.

10.4 Professional Ethics Codes

Security professionals should be guided by established ethics codes:

- **EC-Council Code of Ethics (CEH):** Keep client information confidential; operate only within authorised scope; do not use knowledge for malicious purposes.
- **OISF Responsible Disclosure:** Minimal impact testing; avoid degradation of services; protect user data.
- **OWASP Testing Guide:** Obtain written permission; minimise impact; document and report findings.
- **(ISC)² Code of Ethics:** Protect society; act honourably, honestly, justly; advance the profession.

Chapter Summary

- ✓ Bluetooth spoofing without authorisation is a criminal offence in virtually every jurisdiction.
- ✓ Always obtain written authorisation specifying exact scope, devices, and testing windows.
- ✓ Bluetooth signals propagate — use RF shielding to prevent unintended testing outside scope.
- ✓ Responsible disclosure protects both the public and the researcher.
- ✓ Adhere to professional ethics codes throughout every engagement.

CHAPTER 11

Defensive Strategies and Hardening

11.1 Bluetooth Security Best Practices

Understanding attack techniques is only half the equation for a security professional. Defenders must translate that knowledge into concrete controls:

Control	Priority	Description
Disable Bluetooth when not in use	Critical	The attack surface is zero when BT is off. Enforce via MDM/GPO.
Use non-discoverable mode	High	Prevents enumeration by passive scanners. Only pair in controlled environments.
Enable Secure Connections Only (SC Only)	High	Reject connections from devices that cannot use Bluetooth Secure Connections (BT 4.2+).
Use SSP with Numeric Comparison or Passkey Entry	High	Avoid Just Works pairing, which provides no MITM protection.
Audit and remove stale pairings	Medium	Old pairings represent dormant attack vectors. Remove unknown devices.
Monitor for duplicate BD_ADDRs	Medium	Two devices with the same address signal spoofing activity.
Implement Bluetooth IDS/IPS	Medium	Tools like BlueAuditor and specialised MDM solutions can detect anomalous BT activity.
Apply firmware updates promptly	High	BlueBorne, KNOB, BIAS, BrakTooth — most were fixed in firmware; unpatched devices remain vulnerable.
Use LE Privacy (BLE)	High	BLE devices should use Resolvable Private Addresses to prevent tracking.
Physically control pairing	Medium	Ensure pairing only happens in secure locations; do not pair in public.

Table 11.1 — Bluetooth Security Hardening Controls

11.2 Detecting Spoofing Attacks

Detecting Bluetooth device spoofing requires anomaly detection capabilities. Key indicators of spoofing activity:

- **Duplicate BD_ADDR:** Two devices advertising the same 48-bit address is technically impossible for legitimate devices. Any WIDS (Wireless IDS) monitoring for this should alert.
- **Rapid identity change:** A BD_ADDR that suddenly appears with a different name or CoD than previously recorded is suspicious.
- **RSSI inconsistency:** A 'new' device with the same BD_ADDR appearing with significantly different signal strength than the original suggests a spoofed clone.
- **OUI mismatch:** The OUI (first 3 bytes of BD_ADDR) should correspond to the expected manufacturer. An iPhone claiming a Raspberry Pi OUI is anomalous.
- **Pairing failures:** Excessive connection attempts or authentication failures may indicate an impersonation attempt (attacker lacks the link key).
- **Bluetooth scan logs:** Continuous logging with bluelog and periodic comparison against a baseline allows administrators to identify new, unexpected devices.

11.3 Bluetooth IDS/IPS Solutions

Solution	Type	Key Feature
BlueAuditor	Commercial BIDS	Real-time device classification; anomaly detection; enterprise management
Kismet (Bluetooth plugin)	Open-source WIDS	Passive monitoring; device fingerprinting; alert on new/anomalous BT devices
Ubertooth One + Kismet	Open-source passive	Physical layer capture; BTLE advertising monitoring; PCAP export
Microsoft Defender for Endpoint	Enterprise MDM	Bluetooth device inventory; policy enforcement; anomaly alerting
Jamf / VMware Workspace ONE	MDM (mobile)	Enforce BT policy on managed devices; remote wipe on compromise
Custom BlueZ monitor script	DIY	Parse bluelog output; alert on duplicate or unexpected BD_ADDRs

Table 11.2 — Bluetooth Detection and Management Solutions

11.4 Enterprise Bluetooth Policy

Organisations should implement a formal Bluetooth Security Policy that addresses:

1. **Device inventory:** All Bluetooth-capable devices (including IoT) must be registered in the asset management system.
2. **Approved profiles:** Only pre-approved Bluetooth profiles (e.g., HFP for headsets) are permitted on corporate devices. Object Push (OPP) should be disabled.

3. **Pairing policy:** Pairing of new devices is only permitted in designated, physically secured areas. 'Just Works' pairing must be disabled where possible.
4. **Guest devices:** Personal Bluetooth devices must not connect to corporate systems without explicit MDM enrolment.
5. **Patch management:** Bluetooth firmware updates must be applied within 30 days of release for devices classified as critical.
6. **Incident response:** Suspected Bluetooth attacks must trigger the organisation's standard IR procedure, including device isolation and log preservation.

Chapter Summary

- ✓ The most effective defence is disabling Bluetooth when not in use.
- ✓ Secure Connections Only mode and SSP Numeric Comparison/Passkey Entry eliminate MITM risk.
- ✓ Detecting spoofing requires baseline logging and anomaly detection for duplicate BD_ADDRs.
- ✓ Enterprise environments need formal Bluetooth security policies enforced via MDM.
- ✓ Continuous firmware patching is non-negotiable given the history of critical Bluetooth vulnerabilities.

CHAPTER 12

Case Studies and Security Lessons

■ NOTE

The following case studies are illustrative scenarios based on documented vulnerability classes. They are presented for educational purposes to demonstrate how Bluetooth spoofing techniques relate to real-world security challenges. All testing depicted is conducted in authorised lab environments.

12.1 Case Study 1: Healthcare Device Impersonation Assessment

Background

A hospital engaged a security team to assess the Bluetooth security of its wireless patient monitoring infrastructure. The environment included bedside glucose monitors, pulse oximeters, and nurse call systems — all communicating via Bluetooth to a central nursing station laptop.

Reconnaissance Phase

```
# Passive scan in authorised lab replica environment
$ sudo bluelog -i hci0 -o /tmp/hospital_scan.log -t 300 -v
# Discovered: GlucoseMonitor-04, BD_ADDR=00:09:DD:12:34:56, CoD=0x000906

# SDP enumeration
$ sudo sdptool browse 00:09:DD:12:34:56
# Found: Health Device Profile (HDP) on RFCOMM Channel 4
# No authentication required on HDP channel (legacy device)
```

Spoofing Test

```
# Authorised test: can a spoofed device feed false data to the nursing station?
$ sudo spooftooph -i hci0 \
-a 00:09:DD:12:34:56 \
-n 'GlucoseMonitor-04' \
-d 0x000906

# Attempt connection to nursing station (in authorised lab replica)
# The nursing station accepted the connection without re-authentication
# Finding: BD_ADDR-based trust with no link-layer authentication
```

Findings and Recommendations

- The nursing station accepted connections from any device presenting a known BD_ADDR — no link key verification was performed.

- All medical devices were operating in Security Mode 1 (no authentication) due to legacy firmware.
- **Recommendation:** Upgrade to Bluetooth Secure Connections; implement link-level authentication; deploy Bluetooth IDS; physically control device pairing.

12.2 Case Study 2: Automotive Bluetooth Assessment

Background

A vehicle security research team (with manufacturer authorisation) assessed the Bluetooth subsystem of an in-vehicle infotainment (IVI) system in a controlled facility. The IVI maintained a pairing list of up to 10 Bluetooth devices.

Key Findings

- The IVI system auto-reconnected to any device matching a stored BD_ADDR without PIN re-entry.
- By spoofing the BD_ADDR of a previously paired phone (obtained during recon with owner consent), the test adapter could initiate an HFP session.
- The auto-reconnect behaviour could theoretically allow an attacker to intercept calls in progress if physical proximity is available.
- **Recommendation:** Implement cryptographic link key verification on reconnection; alert driver on new/unexpected device connections; use BLE Privacy for non-HFP profiles.

12.3 Case Study 3: Smart-Home Reconnaissance

Background

A smart-home security audit (residential environment, owner authorised) revealed several Bluetooth LE devices operating without privacy features:

- A smart door lock advertising its BD_ADDR permanently, enabling physical tracking.
- A Bluetooth thermostat using Just Works pairing — any device in range could pair and modify temperature settings.
- A baby monitor streaming audio over BLE with no encryption (Security Mode 2, Level 1).
- **Recommendation:** Enable LE Privacy (Resolvable Private Addresses) on all BLE devices; enforce LE Secure Connections with Numeric Comparison; encrypt all health and safety-critical data streams.

12.4 Lessons Learned

Lesson	Technical Detail	Mitigation
--------	------------------	------------

BD_ADDR is not a secret	It is broadcast in plaintext during every inquiry and advertisement	Never use BD_ADDR alone as an authentication credential
Legacy devices persist	Medical, industrial, and automotive BT devices often run firmware 5-10 years old	Network segmentation; firmware update programme; IDS monitoring
Just Works is dangerous at scale	IoT devices default to Just Works for usability; this means no MITM protection	Require Secure Simple Pairing with user confirmation for critical devices
CoD can be misleading	Any device can claim any CoD; do not make security decisions based on device class alone	Cryptographic identity verification; certificate pinning
Bluetooth IDS is underdeployed	Most organisations have no visibility into their Bluetooth environment	Deploy passive Bluetooth monitoring; build a device inventory

Table 12.1 — Key Lessons from Bluetooth Security Assessments

Chapter Summary

- ✓ BD_ADDR-based trust without cryptographic verification is a systemic vulnerability across industries.
- ✓ Legacy firmware on embedded devices remains the most persistent Bluetooth security risk.
- ✓ Just Works pairing should be considered insecure for any safety-critical or data-sensitive application.
- ✓ Visibility into the Bluetooth environment (IDS, logging) is the first step to effective defence.
- ✓ Authorised testing using spoofing tools provides evidence that drives meaningful security improvements.

APPENDIX A

Troubleshooting Guide and FAQs

Issue: Spooftooph fails with 'Operation not permitted'

Cause: You must run Spooftooph as root or with sudo. BlueZ HCI sockets require CAP_NET_ADMIN and CAP_NET_RAW capabilities.

Fix:

```
$ sudo spooftooph -i hci0 -a AA:BB:CC:DD:EE:FF
```

Issue: 'No such device' or 'hci0: error -16'

Cause: The HCI interface is not up or does not exist. Verify with hciconfig and bring it up.

Fix:

```
$ sudo hciconfig hci0 up
$ hciconfig -a
```

Issue: BD_ADDR does not change after spooftooph runs

Cause: The adapter chipset may not support vendor-specific BD_ADDR write commands. Test with the bdaddr utility. Some Intel adapters require a specific vendor command not implemented in Spooftooph.

Fix:

```
$ sudo bdaddr -i hci0
$ sudo bdaddr -i hci0 AA:BB:CC:DD:EE:FF
```

Issue: Name or CoD change is lost after reboot

Cause: Device name and CoD are volatile. To persist them, set them in /etc/bluetooth/main.conf or run Spooftooph from a systemd service at boot.

Fix:

```
Name = MyDevice
Class = 0x002540 # in /etc/bluetooth/main.conf
```

Issue: 'Scanning...' hangs and returns no devices

Cause: Ensure the target device is in discoverable mode (PSCAN + ISCAN). Check that the adapter is not blocked by rfkill.

Fix:

```
$ sudo rfkill list
$ sudo rfkill unblock bluetooth
```

Issue: bluetoothd conflicts with Spoonftooth

Cause: bluetoothd can interfere with direct HCI access. Stop it temporarily for testing, or use the -S flag to avoid adapter conflicts.

Fix:

```
$ sudo systemctl stop bluetooth
# Run spoonftooth
$ sudo systemctl start bluetooth
```

Issue: Spoonftooth cannot find libbluetooth.so

Cause: The BlueZ development library is not installed. Install it before compiling from source.

Fix:

```
$ sudo apt install libbluetooth-dev -y
```

Issue: Cannot install spoonftooth on Ubuntu

Cause: Spoonftooth is in the Kali repository. On Ubuntu, build from source or use the Kali repository in an isolated VM only.

Fix:

```
$ git clone https://github.com/tw0phi/spoonftooth.git
$ cd spoonftooth && make && sudo make install
```

Frequently Asked Questions

Q: Does Spoonftooth work on all Bluetooth adapters?

A: No. BD_ADDR modification requires vendor-specific HCI commands supported by the controller firmware. CSR8510, many Realtek, and some Broadcom adapters support it. Intel and Qualcomm Atheros adapters may not. Always test with 'bdaddr -i hciX' first.

Q: Will the spoofed BD_ADDR survive a reboot?

A: For most adapters, vendor-commanded BD_ADDR changes are written to volatile memory and reset on power cycle. Some CSR adapters allow permanent writes. To ensure the spoofed address persists across sessions, run Spooftooph from a startup script.

Q: Can Spooftooph be detected?

A: In a monitored environment, yes. A Bluetooth IDS that baselines BD_ADDR → device fingerprint mappings will flag an inconsistency. The HCI commands Spooftooph issues are also visible to btmon on the local system.

Q: Can Spooftooph bypass Bluetooth authentication?

A: No. Spooftooph changes identity parameters only. It cannot replicate a stored link key. If a target device performs full authentication (verifies the link key), spoofing the BD_ADDR alone will not grant access.

Q: Is Spooftooph legal to use?

A: Only with explicit written authorisation from the owner of the devices and wireless environment being tested. Unauthorised use constitutes a criminal offence in most jurisdictions. See Chapter 10.

Q: Can I use Spooftooph with VirtualBox/VMware?

A: Yes, but the USB Bluetooth adapter must be passed through to the guest VM. Native VM Bluetooth (VMware's virtual BT) does not provide HCI-level access needed for spoofing.

APPENDIX B

Glossary of Bluetooth and Cybersecurity Terms

A2DP (Advanced Audio Distribution Profile)

A Bluetooth profile enabling high-quality audio streaming, typically used for wireless headphones and speakers.

ACL (Asynchronous Connection-Less)

The primary data channel in Bluetooth Classic, supporting asynchronous packet transmission.

AFH (Adaptive Frequency Hopping)

A Bluetooth 1.2+ mechanism that avoids interference by dynamically skipping busy frequency channels.

BD_ADDR (Bluetooth Device Address)

A unique 48-bit hardware address assigned to every Bluetooth device, structurally equivalent to an Ethernet MAC address.

BLE (Bluetooth Low Energy)

A power-efficient variant of Bluetooth introduced in version 4.0, targeting IoT and sensor applications.

BlueZ

The official Bluetooth protocol stack for Linux, providing both kernel-space drivers and user-space daemons.

Bluebugging

A Bluetooth attack enabling full control of a vulnerable device's AT command interface via RFCOMM.

BlueBorne

A set of 8 critical Bluetooth vulnerabilities (2017) enabling RCE and MITM attacks without user interaction.

Bluejacking

Sending unsolicited messages to discoverable Bluetooth devices via the Object Push Profile.

Bluesnarfing

Unauthorised access to a Bluetooth device's data (contacts, calendar) via OBEX without authentication.

BrakTooth

A set of 22 Bluetooth vulnerabilities (2021) in commercial SDKs affecting BR/EDR Classic devices.

CoD (Class of Device)

A 24-bit field broadcast by Bluetooth devices to indicate device type (phone, PC, headset) and service capabilities.

ECDH (Elliptic Curve Diffie-Hellman)

A key exchange algorithm used in Bluetooth SSP and BLE Secure Connections to establish shared secrets over an insecure channel.

FHSS (Frequency Hopping Spread Spectrum)

A modulation technique used by Bluetooth Classic to hop between 79 channels 1,600 times per second.

GATT (Generic Attribute Profile)

The BLE application-layer protocol defining how data is structured and exchanged as attributes (characteristics).

HCI (Host Controller Interface)

The standardised command/event interface between Bluetooth host software and controller hardware. Each adapter appears as hciN in Linux.

HFP (Hands-Free Profile)

A Bluetooth profile enabling telephone call handling via a car kit or wireless headset.

HID (Human Interface Device)

A Bluetooth profile for input devices such as keyboards and mice.

ISCAN (Inquiry Scan)

An HCI setting that makes a device respond to Bluetooth inquiry requests — necessary for discoverability.

KNOB Attack

A 2019 vulnerability (CVE-2019-9506) allowing reduction of Bluetooth encryption key entropy to 1 byte, enabling brute-force decryption.

L2CAP (Logical Link Control and Adaptation Protocol)

A Bluetooth layer providing multiplexing, segmentation, and reassembly for higher-layer protocols.

LMP (Link Manager Protocol)

A Bluetooth layer responsible for link setup, authentication, encryption negotiation, and power control.

LTK (Long-Term Key)

A BLE cryptographic key stored after pairing, used to re-establish encrypted sessions on reconnection.

MITM (Man-in-the-Middle)

An attack where an adversary secretly relays and potentially modifies communications between two parties.

OUI (Organizationally Unique Identifier)

The first 3 bytes of a MAC or BD_ADDR, identifying the device manufacturer and assigned by IEEE.

Piconet

A Bluetooth network topology with one master and up to seven active slave devices synchronised to the master's clock.

PSCAN (Page Scan)

An HCI setting that makes a device connectable — required for accepting incoming Bluetooth connections.

RFCOMM

A Bluetooth protocol emulating RS-232 serial communication over L2CAP, used by SPP, HFP, and other profiles.

Scatternet

Multiple interconnected Bluetooth piconets sharing bridge devices that participate in more than one piconet.

SDP (Service Discovery Protocol)

A Bluetooth protocol enabling devices to discover services and profiles available on a peer device.

Spooftooph

A Kali Linux command-line utility for modifying the local Bluetooth adapter's BD_ADDR, device name, and Class of Device for security testing purposes.

SSP (Secure Simple Pairing)

The Bluetooth 2.1+ pairing method using ECDH for key exchange, with four user-interaction models (Numeric Comparison, Passkey Entry, Just Works, OOB).

SweynTooth

A set of 13 BLE vulnerabilities (2019-2020) in commercial SDK implementations affecting BT 4.6–5.3 devices.

Ubertooth One

An open-source hardware platform for Bluetooth physical-layer monitoring and analysis, capable of passive capture of BT Classic and BLE traffic.

Vendor-Specific HCI Command

A non-standard HCI command (OGF=0x3F) defined by a controller manufacturer for features not in the Bluetooth specification, such as BD_ADDR write.

APPENDIX C

References, Citations, and Further Reading

Standards and Specifications

- Bluetooth SIG. (2023). *Bluetooth Core Specification 5.4*. Bluetooth Special Interest Group.
<https://www.bluetooth.com/specifications/specs/>
- IEEE. (2023). *IEEE MA-L (OUI) Public Listing*.
<https://regauth.standards.ieee.org/standards-ra-web/pub/view.html#registries>
- NIST. (2017). *SP 800-121 Rev. 2: Guide to Bluetooth Security*. National Institute of Standards and Technology.
- ETSI. (2022). *ETSI TS 133 328: Bluetooth Security Architecture for 3GPP systems*.

Vulnerability Research

- Armis Security. (2017). *BlueBorne: A New Class of Airborne Attacks*. Armis Research. CVE-2017-0781 through CVE-2017-0785.
- Antonioli, D., Tippenhauer, N., & Rasmussen, K. (2019). *The KNOB is Broken: Exploiting Low Entropy in the Encryption Key Negotiation of Bluetooth BR/EDR*. USENIX Security 2019. CVE-2019-9506.
- Antonioli, D., Tippenhauer, N., & Rasmussen, K. (2020). *BIAS: Bluetooth Impersonation Attacks*. IEEE S&P; 2020. CVE-2020-10135.
- Wu, J. et al. (2020). *BLESA: Spoofing Attacks against Reconnections in Bluetooth Low Energy*. WOOT 2020. CVE-2020-9770.
- Garbelini, M. et al. (2021). *BrakTooth: Causing Havoc on Bluetooth Link Manager*. USENIX Security 2021.
- Garbelini, M. et al. (2020). *SweynTooth: Unleashing Mayhem over Bluetooth Low Energy*. USENIX ATC 2020.

Books and Textbooks

- Jasek, S. (2004). *Hacking Exposed Wireless*. McGraw-Hill / Osborne.
- Wright, J., & Cache, J. (2015). *Hacking Exposed Wireless, 3rd Edition*. McGraw-Hill Education.
- Sanfilippo, L. (2020). *Bluetooth Security Attacks: Comparative Analysis, Attacks, and Countermeasures*. Springer.
- Messier, R. (2014). *CEH v8: Certified Ethical Hacker Version 8 Study Guide*. Sybex.

- Kennedy, D., O'Gorman, J., Kearns, D., & Aharoni, M. (2011). *Metasploit: The Penetration Tester's Guide*. No Starch Press.

Online Resources and Tools

- BlueZ Project. *BlueZ — Official Linux Bluetooth Protocol Stack*. <http://www.bluez.org/>
- Kali Linux Tools. *Spoonftooth — Bluetooth Spoofing Tool*. <https://www.kali.org/tools/spoonftooth/>
- Dunning, JP. *Spoonftooth Source Code*. <https://github.com/tw0phi/spoonftooth>
- Ryan, M. *GATTacker — BLE MITM Proxy*. <https://github.com/securing/gattacker>
- Ostertag, M. et al. *Bettercap — Network Attack Framework*. <https://www.bettercap.org/>
- Bluetooth SIG Security Working Group. *Bluetooth Security Resources*. <https://www.bluetooth.com/learn-about-bluetooth/key-attributes/bluetooth-security/>
- OWASP. *Mobile Security Testing Guide (MSTG) — Bluetooth Security Testing*. <https://owasp.org/www-project-mobile-security-testing-guide/>
- Ubertooth Project. *Ubertooth One Documentation*. <https://github.com/greatscottgadgets/ubertooth>

Further Reading and Certifications

Readers seeking to deepen their expertise in wireless and Bluetooth security may benefit from the following study paths:

- **CEH (Certified Ethical Hacker)** — EC-Council. Covers Bluetooth attacks in the wireless module.
- **OSCP (Offensive Security Certified Professional)** — OffSec. Comprehensive hands-on penetration testing certification.
- **GAWN (GIAC Assessing and Auditing Wireless Networks)** — SANS/GIAC. Dedicated wireless security certification.
- **OWASP IoT Security Testing Guide** — Free resource covering BLE security testing methodology.
- **SANS FOR585** — Mobile Device Forensics, includes Bluetooth evidence sources.
- **Bluetooth SIG Training Portal** — Free online courses on Bluetooth technology and security.

This book was written as an educational resource for the cybersecurity community. Knowledge is most powerful when shared responsibly. Use it to make the world safer.

— *Tanish Sareen, CT University, 2024*